

Automated Relevance Labelling Using an Ensemble of Large Language Models

Development and Evaluation of a Pipeline for Generating Relevance Judgements Using Ensembles of Large Language Models

submitted by

Julian Hergoss

to the Department VI - Computer Science and Media
of the Berlin University of Technology
for the attainment of the academic degree

Bachelor of Science (B. Sc.)

in the degree program

Medieninformatik

Date of submission: January 27, 2026



Studiere Zukunft

Reviewer

Prof. Dr. Siamak Haschemi

Prof. Dr. Alexander Löser

Abstract

Relevance labels are essential for evaluating search systems, but manual assessment is costly and time-consuming. This thesis investigates the feasibility of using large language models (LLMs) to automate the relevance labelling process. In particular, it compares an ensemble-based approach that aggregates the outputs of multiple smaller models with a state-of-the-art single LLM.

The thesis presents the design and implementation of a modular four-stage relevance labelling pipeline based on a particularly flexible software architecture that allows for straightforward exchange of key components such as the provider, the model, or the prompt. The pipeline is capable of interpreting the output of a single LLM or combining the outputs of multiple LLMs using different aggregation strategies, and it supports evaluation against ground-truth relevance judgements.

Using this pipeline, the system is evaluated on the LLM Judge Challenge benchmark by comparison with other LLM-as-a-Judge approaches and human relevance judgements using established inter-rater reliability metrics. The strongest ensemble configuration is ranked 19/46 according to Cohen’s κ and 18/46 according to Krippendorff’s α , placing it close to the median of challenge submissions. In contrast, the state-of-the-art reference model (GPT 5.1) is ranked 13/46 by Cohen’s κ and 8/46 by Krippendorff’s α , corresponding to upper-quartile placement. Despite this gap, the ensemble reduces inference cost by approximately 81% while maintaining similar end-to-end latency.

Two of the initial hypotheses are disproven. First, in the evaluated setting, ensemble aggregation is associated with an increased rate of severe misclassifications, driven by a tendency to overestimate relevance. Second, increasing the number of ensemble members beyond four does not yield significant improvements in label quality. Nevertheless, aggregation strategy is found to play a measurable role in determining label quality. Majority voting with average-based tie-breaking emerges as a reasonable default, although no universally optimal strategy emerges.

Overall, the thesis concludes that an ensemble of smaller models, while not matching the fine-grained accuracy of a top-tier single model, constitutes a technically feasible and economically attractive alternative for production environments prioritizing cost efficiency and robustness.

Contents

1	Introduction	1
1.1	Problem Statement and Research Objectives	1
1.2	Research Questions	2
1.3	Thesis Outline	3
2	Background and Related Works	5
2.1	Introduction to Large Language Models	5
2.2	Introduction to Information Retrieval	8
2.3	Evaluation of Search Systems with Large Language Models	10
2.4	Ensembles of Large Language Models	12
2.5	Criticism of Large Language Models for Search Evaluation	13
3	System Design and Implementation	14
3.1	Requirements Analysis	14
3.2	Software Architecture & Design	16
3.3	Pipeline Implementation	22
3.4	Testing	29
4	Methodology	32
4.1	Underlying Dataset	32
4.2	Model Selection	34
4.3	Prompt Selection	38
4.4	Experimental Design	39
4.5	Evaluation Metrics	40
5	Experimental Results	47
5.1	Run Statistics and Reproducibility	47
5.2	LLM Judge Challenge Ranking	48
5.3	Cost-Latency Trade-offs	50
5.4	Ensemble Size and Aggregation Strategies	51
5.5	Misclassification Analysis	53
6	Discussion	55
6.1	Interpretation of Results	55
6.2	Limitations	58
6.3	Practical Implications	59
6.4	Future Work	62
7	Conclusion	63
	Bibliography	65



List of Figures

2.1	Architecture of a GPT decoder transformer network. Illustration and description adapted from Bishop and Bishop [3].	6
2.2	Illustration of attention depicting influence of words to each other through stroke thickness. Illustration and description adapted from Bishop and Bishop [3].	7
2.3	Illustration of the processing pipeline in a typical TREC track. Illustration adapted from Voorhees [41].	9
3.1	Ports & Adapters architecture, as exemplified in the INFER CLI. Some adapters are included only to demonstrate the flexibility of this architectural pattern and have not been implemented. Illustration adapted from Cockburn [8].	17
3.2	High-level object flow diagram of the pipeline.	18
3.3	ER diagram of the INGEST pipeline in crow's foot notation. The dotted line indicates the boundary to the INFER pipeline.	19
3.4	ER diagram of the INFER pipeline in crow's foot notation	20
3.5	ER diagram of the AGGREGATE pipeline in crow's foot notation. The dotted line indicates the boundary to the INFER pipeline	21
3.6	Directory structure of the INFER module. The folder structure divides the module into the adapter, application, domain, and startup layer.	24
3.7	Illustration of the testing architecture. The application core is surrounded by mocked adapters that simulate external dependencies such as input, I/O, provider, prompt, and parsing components. Illustration adapted from Cockburn [8].	30
5.1	Cohen's κ for each model across repeated runs, computed against human annotations.	48
5.2	Agreement between individual models, the ensemble, and the reference model with human annotations, measured using Cohen's κ . Individual models are sorted from left to right by parameter count.	50
5.3	Total cost and latency for the reference model and the ensemble.	51
5.4	Cohen's κ as a function of ensemble size when incrementally adding bigger models, starting from the smallest model.	52
5.5	Cohen's κ as a function of ensemble size when incrementally adding smaller models, starting from the largest model.	52
5.6	Heatmap of ensemble-based relevance labels (MVA) compared to human labels.	53
5.7	Heatmap of GPT 5.1 relevance labels compared to human labels.	54



List of Tables

2.1	Example ranking for the query “What is Python?” with relevance judgements and DCG@4.	8
2.2	Comparison of industrial LLM-based relevance labeling systems. Microsoft reports improvements relative to human labellers using searcher-derived ground truth as reference, while Zalando compares LLM annotations to third-party human assessors via inter-annotator agreement. Reported values are approximate.	11
3.1	Command-line options for the INGEST CLI	23
3.2	Information contained in the ingest run summary.	23
3.3	Command-line options for the INFER CLI	24
3.4	Information contained in the infer run summary.	25
3.5	Command-line options for the AGGREGATE CLI	27
3.6	Example of incompatible judgements that must not be aggregated	28
3.7	Command-line options for the EVALUATE CLI	28
4.1	LLM Judge Challenge Dataset Statistics (Test Split). Only unique documents that were rated are included.	32
4.2	Label Distribution for the LLM Judge Challenge	32
4.3	TREC relevance scale for passages.....	33
4.4	Single-model baselines. Inference costs are reported in USD. Per-sample costs are estimated from a 100-sample smoke test and extrapolated to the full evaluation set of 4,423 judged query-document pairs (see table 4.1).	35
4.5	Model filtering steps applied to the OpenRouter catalog.	36
4.6	Ensemble candidates that satisfied the operational constraints. Sorted by parameter size.	37
4.7	Confusion matrix for two raters with K relevance classes	42
4.8	Interpretation of Cohen’s κ according to Landis and Koch [16].	43
4.9	General form of a reliability data matrix following Krippendorff [15] notation.	44
4.10	General form of a coincidence matrix.	44
4.11	Example reliability data matrix for relevance labelling in this thesis.	44
4.12	Coincidence matrix for the six-document example (unweighted pair counts).	45
4.13	Weights computed with $\text{ordinal}\delta_{ck}^2$ for the six-document example.	45
4.14	Interpretation of Krippendorff’s α	46
5.1	Intra-model agreement across repeated runs measured with Krippendorff’s α	47
5.2	LLM Judge Challenge rankings with our approaches highlighted.	49
5.3	Confusion Matrix Comparison: Ensemble (MVA) vs GPT-5.1	54



Listings

3.1	Core inference loop of the application layer coordinating prompt construction, LLM invocation, parsing, and incremental persistence. Logging and other small details have been removed for brevity.	26
3.2	Example test for the inference pipeline. Core application components, including the prompt builder and response parser, are instantiated using their real implementations. External dependencies such as the LLM provider and file system interactions are replaced with test doubles. This test is able to cover the INFER pipeline almost end to end and verifies the core workflow runs as intended.	30
4.1	Prompt adapted from Thomas et al. [37] to the TREC relevance scale. . . .	38



Abbreviations

ACM Association for Computing Machinery 10

AV Average Voting 39, 48, 51, 58

CLI command-line interface 22, 23, 25, 27, 28

IR information retrieval 2, 8–11, 14, 15, 22, 23, 33

IRRJ Information Retrieval Research Journal 13

LLM large language model 1–3, 5, 8, 10–19, 23, 25, 29, 33–35, 38, 41, 43, 47, 56, 59, 60, 63

LLM4Eval LLMs for automated evaluation 11

MS Marco Microsoft Machine Reading Comprehension 32–34

MVA Majority Voting with Average Tie Breaking 39, 48, 51, 53, 55, 58, 60

MVR Majority Voting with Random Tie Breaking 39, 48, 51, 55, 58

NIST National Institute of Standards and Technology 9, 10, 33, 34

SIGIR Special Interest Group on Information Retrieval 10, 11

TREC Text Retrieval Conference 5, 9, 32–34, 40



Glossary

few-shot prompting a prompting technique where the model is given a small number of input–output examples along with the task description to guide its behavior on new inputs 11

hard disagreement a disagreement between two relevance judgements where one rater assigns a low relevance label and the other assigns a high relevance label following Thomas et al. [37]. In a confusion matrix, hard disagreements correspond to the top-right and bottom-left corner cells. 53, 56, 57, 59

inter-rater reliability a measure of the degree to which independent raters assign consistent scores or labels to the same set of items, used to assess the quality and reproducibility of annotations 28, 41, 43, 51, 57, 58

LLM Judge Challenge an evaluation benchmark for assessing large language models as automated relevance judges, based on the TREC Deep Learning Track dataset [31] iii, 3, 11, 29, 32–34, 36, 38, 39, 47, 48, 55, 56, 63

one-shot prompting a prompting technique where the model is given exactly one example of the task along with the task description, helping it generalize the pattern to new inputs 11

Ports & Adapters architecture a software architecture pattern developed by Alistair Cockburn that separates core application logic from external systems by defining explicit ports and adapter interfaces 16, 17, 29

qrel short for query relevance judgment; a relevance judgment mapping a query to a document with an assigned relevance label. Synonyms: relevance judgment, relevance label, relevance score, relevance annotation. 11, 18, 34, 58

soft disagreement a disagreement between two relevance judgements where the assigned labels differ by one adjacent relevance level. In a confusion matrix, soft disagreements appear in cells adjacent to the main diagonal. 53, 56

token a basic unit of text processed by a language model, such as a word, subword unit, punctuation mark, or special symbol 6

Transformer a neural network architecture based on self-attention mechanisms for processing sequences in parallel, originally introduced for machine translation 5



Chapter 1

Introduction

*donista*¹ is a small but ambitious non-profit online shopping platform that transforms affiliate commissions from partner shops into donations for charitable organizations. Before shopping, users can choose a charity to support, and at least 50% of the commissions generated through donista are donated directly to their chosen cause.

During my internship at donista, I had the opportunity to work alongside a group of talented engineers who develop the platform in their free time as a voluntary initiative. At the time of writing this thesis, the platform is less than a year old and in active development. Naturally, some features need refinement - one of them being the search functionality. The search bar, a key component of most online shopping platforms, was at that time implemented as a basic keyword search, intended primarily to establish baseline functionality. Therefore, the search system required substantial revision.

1.1 Problem Statement and Research Objectives

While developing an improved search functionality, it quickly became evident that progress could not be meaningfully assessed without a way to measure the system's performance. The key metric for such evaluation is the relevance of search results to the user's query. However, manually assessing relevance is not feasible. *donista's* employees cannot reasonably review and label large quantities of search queries and their results - especially not repeatedly throughout the iterative development of the search system.

With recent advances in large language models (LLMs), it became evident that relevance labeling could potentially be automated. Work from Microsoft and Zalando reports promising results, showing that LLM-based relevance assessment can be orders of magnitude faster and cheaper than human annotation, while the resulting labels exhibit quality comparable to or exceeding human judgements [37, 13]. These findings provided a key motivation for investigating LLM-based relevance labeling in this thesis.

The central problem addressed in this thesis is therefore how to leverage LLMs to automate the evaluation of search result relevance, enabling scalable and efficient comparison between search system versions. The objective of this thesis can be further divided into two distinct aspects.

¹<https://donista.org/>



First, this thesis evaluates the feasibility of using LLMs, and in particular ensemble-based approaches, for automated relevance assessment in the context of donista. The primary objective is to determine whether such methods can be applied reliably in a practical production setting. Where applicable, the analysis aims to derive concrete recommendations for deployment, including guidance on model selection, ensemble size, and aggregation strategy.

Second, the thesis examines LLM-based relevance assessment beyond this specific application context. While the experimental design is motivated by the needs of donista, the reported results are analyzed using established evaluation metrics and are framed to allow comparison with related work. In this sense, the thesis also serves as an exploratory study of ensemble-based relevance evaluation and may provide insights that are of interest to the broader information retrieval (IR) research community.

1.2 Research Questions

Based on the stated objectives, the following primary and secondary research questions are formulated. Both the primary and secondary research questions are addressed explicitly in the discussion section of Chapter 6, with the secondary questions providing additional context to support the interpretation of the primary results.

Primary Research Questions

(RQ1) Can an ensemble of smaller LLMs produce labels of quality comparable to that of a single state-of-the-art model?

Smaller language models are generally expected to produce less accurate relevance judgments than large, state-of-the-art models due to their reduced capacity. An ensemble approach may partially offset this limitation by aggregating multiple independent judgments, potentially compensating for individual model errors and misunderstandings. This research question therefore evaluates whether such ensembles can match the relevance labelling quality of a single large model, as measured by agreement with human judgements and the frequency of severe rating discrepancies.

(RQ2) How do ensembles of smaller LLMs compare to a single state-of-the-art model in terms of inference cost and latency?

Beyond labelling quality, the practical viability of automated relevance evaluation systems depends on their computational efficiency. While smaller models are typically cheaper to run and easier to deploy, combining multiple models in an ensemble increases the total number of inference calls and may substantially affect runtime. This research question examines the cost and latency implications of ensemble-based relevance labelling and assesses whether the resulting overhead remains acceptable.

Secondary Research Questions

(SRQ1) Can a diverse ensemble of LLMs reduce the number of hard disagreements in comparison to a single state-of-the-art LLM for relevance labeling?



Thomas et al. [37] observe that hard disagreements most often arise when an LLM misunderstands the underlying query. If hard disagreements are primarily caused by model-specific misunderstandings of the query, then different LLMs are unlikely to exhibit the same misunderstandings. Aggregating relevance judgements of multiple LLMs should therefore reduce the number of hard disagreement compared to that of a single LLM.

(SRQ2) Does increasing ensemble size lead to more stable relevance judgements through increased consensus among models?

If individual LLMs produce partially independent relevance judgements, then aggregating a larger number of models increases the likelihood that model-specific errors are outweighed by shared consensus. Under this assumption, smaller ensembles are more likely to reflect individual model errors, whereas larger ensembles should converge toward more stable relevance labels.

(SRQ3) How does the choice of aggregation strategy affect inter-rater reliability?

Different aggregation strategies can yield different final labels from the same set of model judgements, particularly when votes are split across relevance classes and ties occur. Aggregation strategies that emphasize consensus among models, such as majority voting, and strategies that emphasize compromise through score averaging, such as average voting, may differ in how well they align with human relevance judgements. This question tests whether one class of aggregation strategy yields consistently higher inter-rater reliability than the others.

1.3 Thesis Outline

Chapter 2 establishes the theoretical foundation underlying this thesis. It introduces LLMs, with a particular focus on the architecture of GPT-based models used throughout the relevance judgement pipeline. The chapter further introduces core terminology and evaluation frameworks from the field of information retrieval and reviews related work to situate the proposed pipeline within the current state of the art.

Chapter 3 presents the design and implementation of the relevance labelling pipeline. It first provides a conceptual overview of the pipeline architecture and then describes the individual components and their implementation in detail. The chapter also outlines the applied testing philosophy and approach, illustrating how tests are used throughout the system to validate critical functionality, like the correct computation of information retrieval metrics.

Chapter 4 describes the experimental methodology used to evaluate the pipeline. It introduces the benchmark dataset used for evaluation and explains how withheld relevance labels from the LLM Judge Challenge are recovered. The chapter further motivates key experimental design decisions, including model and prompt selection, documents all relevant configurations, and presents the evaluation metrics together with their computation and interpretation guidelines.

Chapter 5 reports the experimental results. It first addresses reproducibility and demonstrates the stability of the obtained results. The chapter then compares the proposed approaches to other submissions to the LLM Judge Challenge and analyzes cost-latency trade-offs as well as the overall behavior of the ensemble in comparison to the reference model.



Chapter 6 discusses the results in the context of the stated primary and secondary research questions. The findings are interpreted in relation to existing work, and their implications for both the information retrieval research community and the donista platform are derived. The chapter also outlines key limitations of the study and reflects on their impact on the reported findings.

All code and data developed in the context of this thesis are publicly available at <https://github.com/julianbht/llm-ensemble>.

Note: Across figures, the colors do not transport any meaning. They are solely for visual clarity.

Tip: Use the colourful squares at the top of each page to quickly navigate to the table of contents.



Chapter 2

Background and Related Works

This chapter provides the theoretical and empirical foundation for the work presented in this thesis. It begins with an overview of LLMs, with a particular focus on Transformer-based architectures and decoder-only models of the GPT family, as these form the basis of the models used throughout this work. The chapter then introduces core concepts of information retrieval, including benchmark-driven evaluation paradigms such as the Text Retrieval Conference (TREC), under which the dataset used in this thesis was produced. Building on this background, the chapter reviews recent research on the use of LLM for search system evaluation, situating the thesis within the current state of the art. Particular attention is given to large-scale industrial applications at Microsoft and Zalando, which demonstrate the practical viability of LLM-based relevance labeling. Finally, the chapter discusses ensemble approaches and critical perspectives on the use of LLMs for evaluation, outlining both their potential and their limitations.

2.1 Introduction to Large Language Models

The field of natural language processing (NLP) has long studied methods that enable computers to communicate like humans through text and speech [44]. Language modelling in particular has been widely studied in the past two decades with the goal of advancing language intelligence in machines [44]. Many different approaches have been studied, evolving from statistical language models to neural language models, to pre-trained language models [44]. In recent years, this line of work has culminated in LLMs, which can be understood as large-scale Transformer-based pre-trained language models [44].

The Transformer was originally proposed as a sequence-to-sequence architecture for machine translation [39]. Subsequent work demonstrated that it achieves state-of-the-art performance across a wide range of tasks [18]. As a result, the Transformer has become the dominant architecture for modern language models. Beyond NLP, Transformer-based models have also been successfully applied in domains such as computer vision, audio processing, and fields including chemistry and the life sciences [18, 3].

Transformer-based language models may contain hundreds of billions of parameters and are trained on massive datasets [3, 44]. The use of large model sizes and training corpora is motivated by empirical scaling laws. The scaling hypothesis states that significant performance improvements can be achieved by increasing the number of model parameters and training on correspondingly large datasets [3, 44]. Thus, the term *large* language model



is used [3].

As established, the Transformer architecture has proven to be highly versatile. Within Transformer-based language models, a common distinction is made between encoder-only, decoder-only, and encoder-decoder architectures [3]. We will focus on decoder-only Transformers, as they form the basis of the Generative Pre-trained Transformer (GPT) family of models [3]. Since these models are used throughout this thesis to generate relevance judgements, it is therefore useful to examine their internal mechanisms in more detail.

The model takes as input the first $n - 1$ tokens and produces a conditional probability distribution over the possible values of token n . By sampling from this distribution, token n is generated. For this reason, the model is referred to as a generative model. By repeatedly applying this procedure, arbitrarily long token sequences can be produced. At each step, the newly generated token is concatenated with the previous input sequence and fed back into the model, forming an *auto-regressive* generation loop [3].

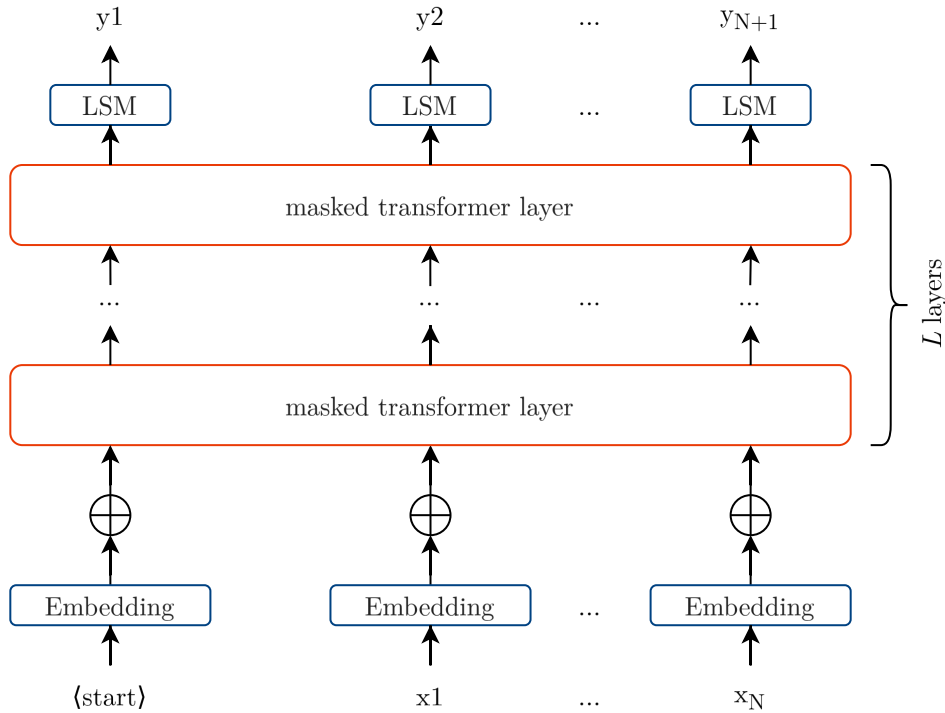


Figure 2.1: Architecture of a GPT decoder transformer network. Illustration and description adapted from Bishop and Bishop [3].

Figure 2.1 depicts the process. First, the input text is decomposed into tokens $x_1, \dots, x_N$. Tokens may correspond to whole words, subword units, punctuation, or structural symbols. Each token is mapped to a numerical representation called an embedding. An embedding is a high-dimensional vector of real numbers, which can be interpreted as a point in a latent space. This space is structured such that semantically similar tokens are located close to each other. In embedding space, direction carries semantic meaning. To give a concrete example, GPT-3 by OpenAI uses embedding vectors with 12,288 dimensions.

The input token sequence is shifted to the right so that the input token x_n corresponds to the output y_{n+1} . Each output y_n represents a probability distribution of the next token. During generation, the distribution may be used to select the next token through



a procedure called sampling. During training, the next token x_{n+1} is known and serves as the target for the output distribution y_n . To prevent the model from exploiting future tokens that are not available at inference time, masked attention is employed. It ensures that each token can attend only to itself and to earlier tokens in the sequence. We will take a look at the inner layers of the transformer and explain the concept of attention shortly.

Positional encoding, depicted in fig. 2.1 by the circle with a cross, is added to each token embedding to provide information about the position of tokens within the sequence. Since the transformer architecture processes tokens in parallel, positional encodings allow the model to distinguish between tokens occurring at different positions. Finally, LSM denotes a linear-softmax output layer that maps each transformer output to a probability distribution over the set of tokens.

Representing words as vectors predates Transformers. The first major architectural innovation introduced by the Transformer is the self-attention mechanism [3, 44]. Self-attention is the fundamental concept that underpins the Transformer architecture [3]. To motivate the need for an attention mechanism, Bishop and Bishop [3] provide the following example:

I swam across the river to get to the other **bank**.
I walked across the road to get cash from the **bank**.

In order to predict the token *bank*, the model must take the preceding context into account. In the second sentence, words such as *cash* indicate a financial context, whereas in the first sentence, tokens like *swam*, *across*, and *river* imply a geographical meaning. Not all tokens contribute equally to this interpretation. For example, the word *get* appears in both sentences but has little influence on the intended meaning. The model must therefore attend more strongly to contextually informative tokens, which motivates the term *attention* [3].

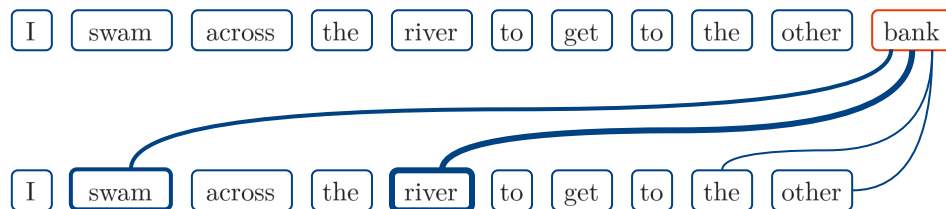


Figure 2.2: Illustration of attention depicting influence of words to each other through stroke thickness. Illustration and description adapted from Bishop and Bishop [3].

For reasons of brevity, we only briefly outline how the self-attention mechanism operates. In short, self-attention allows the embedding vectors to update each other based on context within the sequence. Each token embedding is decomposed into a query, key, and value representation. These representations are compared across all tokens, allowing each token to be updated based on its relationship to the others in the sequence.

Based on this perspective of attention, a transformer layer can be viewed as a function that enriches a given embedding vector by mapping it to a representation that incorporates information from the other embedding vectors in the sequence [3]. As illustrated in fig. 2.1, multiple such layers may be stacked, with the aim of forming increasingly rich representations of the semantic content within the sequence [3]. We have not mentioned so far that each transformer layer also contains its own weights and biases. Among these are the parameters of a feed-forward network, which are initialized randomly and learned from data during the training process. This feed-forward network enables each token representation to be



transformed independently, allowing the model to refine and reshape the information it carries.

Before concluding this section, we briefly clarify the terminology used throughout this work. We refer to the members of the ensemble as small LLMs, which at first glance appears to introduce an oxymoron. In the literature, LLMs are informally described as such once they reach a parameter count on the order of ten billion [44]. In this work, however, we also use the term LLMs to refer to models with fewer parameters. Although some works introduce the term *small language model*, this terminology is not well established and is therefore avoided here. Instead, we use LLMs to denote a class of language models that exhibit emergent behaviors associated with scaling laws, independent of their exact parameter count.

2.2 Introduction to Information Retrieval

Generally speaking, the goal of an IR system is to satisfy an information need by searching a collection of texts and returning items that are relevant¹. Relevance to the information need is the central criterion for evaluating IR system performance. However, natural language is complex which makes the task of determining relevancy in a programmatic manner inherently difficult.

A foundational evaluation framework for IR emerged in the 1960s through the work of Cyril Cleverdon at Cranfield University in the United Kingdom. This framework, known as the *Cranfield method* or *Cranfield paradigm*, established core concepts that still define IR terminology today: *documents* as the items to be retrieved, *topics* as formal representations of information needs, and *relevance judgements* as subjective assessments of how well a document satisfies a topic [41].

Using these concepts, an IR system can be evaluated in a structured way. For a given topic, the order in which the IR system returns the documents explicitly defines a ranking. This ranking is analyzed by examining the relevance judgements assigned to the retrieved documents. A system performs well if it ranks relevant documents above irrelevant ones and poorly if it does the opposite. By assigning a numerical metric to each ranking, different IR systems operating on the same set of topics and documents can be compared [9, 41].

Rank	Retrieved Document	Relevance	DCG@4
1	Python is a popular programming language	3	3.00
2	History of the Python snake	0	3.00
3	Python tutorials for beginners	2	4.26
4	Monty Python comedy group	1	4.56

Table 2.1: Example ranking for the query “What is Python?” with relevance judgements and DCG@4.

As an illustrative example of ranking-based evaluation in IR, we briefly introduce Discounted Cumulative Gain (DCG). DCG is commonly used in IR to measure the quality of a ranked list of documents by accounting for both the relevance of documents and their positions

¹In principle, data in information retrieval is not limited to text and may include images, videos, or other formats; however, this work focuses exclusively on textual data.



in the ranking [14]. Highly relevant documents appearing early in the ranking contribute more to the overall score than equally relevant documents appearing at lower ranks.

Several equivalent variants of DCG exist in the literature. Following common practice in modern IR evaluations, this work uses the exponential gain formulation with a logarithmic discount [14]. Given a ranking of documents for a query, DCG at rank k is defined as

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad (2.1)$$

where rel_i denotes the relevance judgement of the document at rank i . The logarithmic discount reduces the contribution of documents retrieved at lower ranks, reflecting the assumption that users are less likely to examine documents further down the ranking.

The National Institute of Standards and Technology (NIST) adopted the Cranfield method and refined it through the development of the TREC. Together, they established a systematic framework for creating large-scale IR datasets that enable researchers to develop, evaluate, and advance IR technologies. Over 300 research groups from 20 countries across 6 continents participate in these events, making them a central component of the IR research landscape [41].

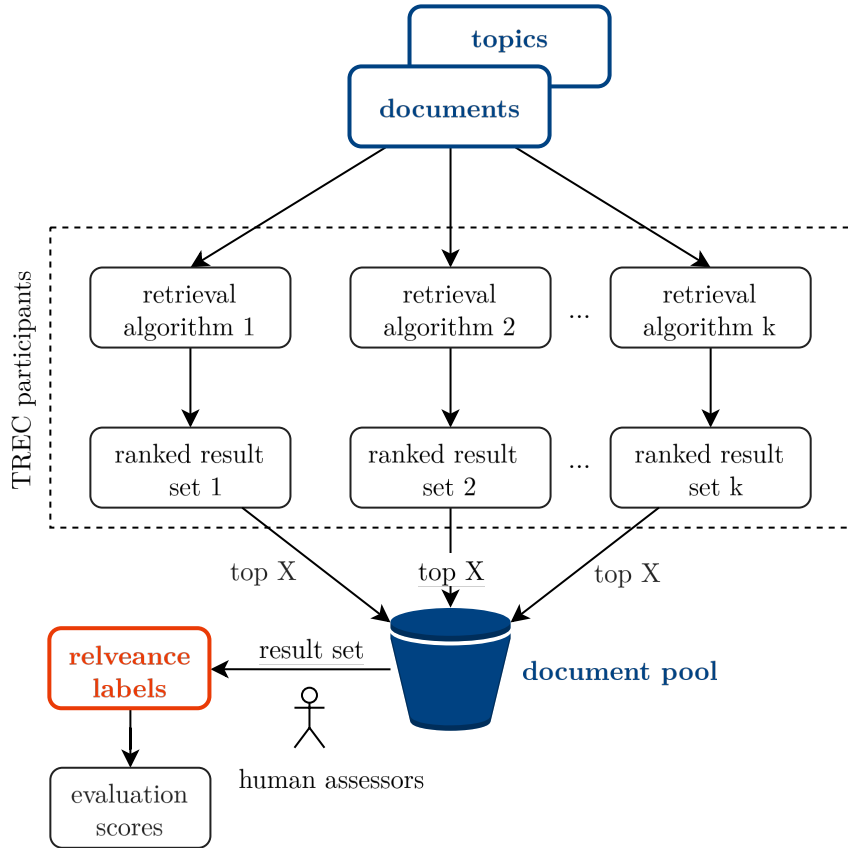


Figure 2.3: Illustration of the processing pipeline in a typical TREC track. Illustration adapted from Voorhees [41].

The dataset used in this thesis was largely developed within the TREC framework. The TREC workshop is organized into multiple tracks, each addressing a distinct retrieval task or research problem. The process of these tracks is depicted in fig. 2.3. For each track, the



organizers first define a set of topics and a document collection. Participating research teams then submit runs, in which their IR systems rank the documents according to estimated relevance for each topic. The top-ranked documents across all submissions are pooled under the assumption that they are likely to represent the most relevant documents. NIST assigns human assessors to these pools, who produce relevance judgements for the documents with respect to the corresponding topics. Finally, all submitted runs are evaluated and compared by computing standard IR metrics over their resulting rankings [4, 41].

2.3 Evaluation of Search Systems with Large Language Models

Generating relevance labels with the help of human assessors is itself a challenging task. To ensure sufficient quality, assessors must be carefully trained, which requires the development of comprehensive guidelines that are both precise and digestible. As a result, human annotation is slow and costly [13, 37].

The complexity of the task is illustrated by the scale of existing guidelines. For example, Google’s *Search Quality Raters Guidelines* comprise 170 pages of densely formulated instructions [19]. Hosseini et al. [13] from Zalando describe an evaluation consisting of 50,000 queries with 20 products per query as “a modest evaluation setup for large-scale IR systems”. They further estimate that conducting a single evaluation of this scale requires them between 5,500 and 8,500 hours of human labor.

Despite these efforts, agreement between human assessors can vary substantially. Hosseini et al. [13] report an inter-annotator agreement of 60.2%, while Zheng et al. [45] find that agreement among expert assessors can reach up to 81%. Given that human assessment is both costly and difficult to scale, the use of LLMs as relevance annotators has emerged as a natural alternative in light of recent advances in model capabilities. In the literature, relevance judgements produced by human assessors are commonly referred to as *gold labels*. By contrast, relevance judgements generated by machines are typically referred to as *silver labels*.

Industrial Adoption of Large Language Models for Search Evaluation

Microsoft. In 2023, Thomas et al. [37] from Microsoft reveal that they have turned to LLM relevance labeling at scale. They presented their study titled *Large Language Models Can Accurately Predict Searcher Preferences* at the Association for Computing Machinery (ACM) Special Interest Group on Information Retrieval (SIGIR) Conference on Research and Development in Information Retrieval. They write that LLMs “have proved more accurate than any third-party labeller, including staff; they are much faster end-to-end than any human judge, including crowd workers; they scale to much better throughput; and of course are many times cheaper.” They were able to compare their automated LLM labelling system with third-party labellers by establishing a separate ground truth. This ground truth was derived from the searchers themselves, under the rationale that the searcher can most accurately assess whether a result satisfies their underlying query intent. They found that their automated labeling system, utilizing GPT-4, was slightly more accurate, ten times faster, and twenty times less expensive.

Zalando. In 2024, Hosseini et al. [13] from Zalando demonstrated the effectiveness of multimodal LLMs for large-scale e-commerce query-product relevance assessment. Their



framework employs a two-stage LLM setup. In the first stage, an LLM analyzes the user’s search query and its context to identify key elements such as brand, color, and product category. Using this information, it generates a query-specific annotation guideline that defines how to assess product relevance. In the second stage, the multimodal annotator LLM evaluates each query–product pair, incorporating textual and visual product descriptions along with the generated query annotation guidelines. It assigns one of three predefined relevance labels: (i) irrelevant, (ii) acceptable substitute, or (iii) highly relevant. They found that their automated system leveraging LLMs is up to 1,000 times cheaper than human labour and significantly faster (20 minutes compared to 3 weeks). In addition, agreement between the LLM and the human majority vote slightly exceeds human inter-annotator agreement, indicating that the automated annotations are not only more efficient but also at least as consistent as human judgements.

Factor	Microsoft	Zalando
Throughput	10× higher	1000× higher
Cost	20× cheaper	100–1000× cheaper
Quality	Higher alignment with ground truth	Slightly higher inter-annotator agreement

Table 2.2: Comparison of industrial LLM-based relevance labeling systems. Microsoft reports improvements relative to human labellers using searcher-derived ground truth as reference, while Zalando compares LLM annotations to third-party human assessors via inter-annotator agreement. Reported values are approximate.

The LLM Judge Challenge

The LLM Judge Challenge [31] gives an overview of the current state of the art in automated LLM-based relevance labeling. The challenge brought together leading research groups from the IR community to evaluate and compare LLM-based relevance labeling methods. In early 2025, the challenge concluded, with eight international teams submitting 42 runs using LLMs to generate qrels (query relevance judgments). Organized as part of the LLMs for automated evaluation (LLM4Eval) workshop at the SIGIR conference in 2024, its goal was to create a large and diverse collection of automatically generated relevance judgments, enabling systematic analysis of biases, model differences, and trade-offs between human and machine assessors. Some particularly well-performing approaches to generating relevance labels using LLMs include:

- Using few-shot prompting (providing a few input–output examples along with the task description). This approach ranked best regarding Krippendorff’s α .
- Using one-shot prompting (providing exactly one example along with the task description) and a specialized prompt based on the works of Thomas et al. [37]. This approach ranked best regarding Cohen’s κ .

We will revisit the LLM Judge Challenge in section 4.1, where its dataset, construction process, and role as the primary evaluation benchmark in this thesis are described in detail.



2.4 Ensembles of Large Language Models

In humans, a diverse group of independently deciding individuals can be more likely to make certain types of decisions and predictions more accurately than single individuals [12, 36]. This phenomenon, known as the *wisdom of the crowd*, provides a fundamental philosophical motivation for employing ensemble methods. Schoenegger et al. [33] demonstrated that individual LLMs underperform relative to human crowds in probabilistic forecasting. However, they also show that an ensemble of LLMs achieves performance that is statistically indistinguishable from aggregated human judgments. They termed this phenomenon *wisdom of the silicon crowd*.

The impact of LLM ensembles on automated relevance labeling systems remains largely unexplored [30]. However, ensemble methods have been extensively studied in classical Machine Learning, where ensemble learning has demonstrated improved robustness and generalization across various tasks [24, 27, 6]. To establish a foundation for adopting LLM ensembles, the following section highlights key ensemble strategies that may inform their design.

Bagging, Boosting and Stacking are popular ensemble techniques in Machine Learning [24, 27]. In stacking, a meta-model integrates the predictions of multiple base models to produce a final output. Bagging involves training multiple models on different subsets of the data and averaging their predictions to reduce variance and mitigate overfitting. Boosting, by contrast, trains models sequentially, with each subsequent model focusing on correcting the errors of its predecessors - a strategy that has proven effective for handling imbalanced datasets.

Another notable ensemble-learning paradigm is the Mixture-of-Experts (MoE) framework, in which an input is dynamically routed to a subset of specialized expert models. This architecture allows efficient computation by activating only the most relevant experts for a given input. Thereby reducing inference costs while maintaining diversity and specialization among models [24].

The underlying idea of LLM Ensembles is to collect a diverse set of judgements, exploiting the strengths of different LLMs while compensating for their individual weaknesses, thereby producing a more balanced and robust evaluation outcome. With this idea in mind, Rahmani et al. [30] introduced *JudgeBlender*, a framework that leverages multiple smaller LLMs to generate relevance judgments. The final assessment is produced by aggregating the individual judgments, typically through majority voting or by computing the average score. This approach is designed to circumvent intra-model biases and has strongly influenced the design of the system developed in this thesis.

A central challenge in such systems lies in the aggregation of multiple LLM-based evaluations. Relying on a single model to act as a meta-judge risks reintroducing the very biases ensemble methods seek to reduce. To address this issue, Verga et al. [40] proposed the Panel of LLM Evaluators framework, in which several smaller and diverse models collaboratively assess outputs. Their individual evaluations are then combined using aggregation functions such as maximum or average voting. The authors show that this method achieves stronger correlations with human judgments than a single large evaluator (GPT-4), while simultaneously reducing intra-model bias and computational cost.



2.5 Criticism of Large Language Models for Search Evaluation

Abels and Lenaerts [1] demonstrate that the selection of LLMs must be undertaken with caution, as inappropriate choices can lead to adverse outcomes. Their analysis of 18 LLMs revealed a tendency for these models to exhibit similar errors. As a result, there is a risk of consensus forming around incorrect conclusions. Insufficient diversity within LLM ensembles can thus amplify pre-existing biases rather than mitigate them. Moreover, Hong and Page [12] provide evidence that debate-style frameworks - where an LLM is instructed to argue for or against potential solutions - may inadvertently intensify bias instead of fostering balanced reasoning.

In 2025, Soboroff [34] published an article titled *Don't Use LLMs to Make Relevance Judgments* in the Information Retrieval Research Journal (IRRJ). Soboroff argues that the source of relevance judgments defines a ceiling on performance, since no evaluation system can be measured as performing better than the ground truth it is compared against. Because information retrieval systems aim to model human information needs, he concludes that humans must define the ideal notion of relevance.

Thomas et al. [37] specify this further: they argue that relevance labels used as ground truth must be determined by the searcher themselves. For example, a third-party assessor may fail to identify navigational queries. Query ambiguity further illustrates this challenge: when searching for "Python", a user might be seeking programming tutorials or information about the snake species. If a third-party assessor - whether human or machine - fails to capture such nuances, fine-tuning a search engine on their labels may prevent it from effectively addressing users' information needs. While Thomas et al. [37] were able to develop an LLM-based system that produces more accurate relevance labels than third-party human annotators, their system ultimately still requires a human-based evaluation framework to assess its own performance and maintain reliability.

In the same year, Dietz et al. [10] detail numerous risks that may negatively impact evaluation when using LLMs. One such risk is the circularity trope, where a LLM is involved both in generating system outputs and in evaluating them, potentially leading to biased or self-reinforcing judgments. Another potential issue is the phenomenon known as test set leakage, which may occur if an LLM used for evaluation has been trained on the same dataset it is assessing. Such overlap could compromise the credibility of the evaluation, as the model may rely on memorized data.

It is well established that LLMs can introduce systematic biases when employed for evaluative tasks [45, 42, 5, 17]. For instance, LLMs are susceptible to position bias, a tendency to disproportionately favor the first option when comparing alternatives [45, 42]. Moreover, Chen et al. [5] examined the overlap between human and LLM biases in relevance assessments and found that both exhibit misinformation oversight and beauty bias, while LLMs additionally display gender and authority bias. Given the likelihood that LLMs harbour further, yet uncharacterised biases, their use in relevance labelling poses a substantial risk of bias propagation.



Chapter 3

System Design and Implementation

This chapter describes the software architecture and implementation of the relevance labelling pipeline. It begins with a concise requirements analysis that captures the conceptual goals and constraints of the system. The chapter then details the applied architectural style, motivated by the need for component interchangeability, and support for systematic experimentation. Finally, it outlines the implementation of the pipeline stages and the applied testing strategy.

3.1 Requirements Analysis

Actors

The system is designed to support two primary actors:

Actor#1: IR researcher. This actor conducts experiments on automated relevance labeling and is primarily concerned with evaluating the impact of prompts, models, model parameters, and aggregation strategies on relevance judgments and evaluation metrics.

Actor#2: donista employee. This actor uses the system to generate relevance labels for the output of a search engine based on query-document pairs. The generated labels and metrics are intended for search quality assessment.

Use Cases

UC#1: IR Research Experimentation. A researcher uses the system to conduct controlled experiments on automated relevance labeling using LLMs. The system supports systematic variation of datasets, prompt templates, model configurations, and ensemble strategies in order to analyze their impact on relevance judgments and evaluation metrics.

UC#2: Relevance Assessment of Search Results. A donista employee uses the system to assess the relevance of search engine results by generating automated relevance judgments for query-document pairs produced by a search engine.



User Stories

US#1: As a researcher, I want to work with different IR datasets in order to analyze how the relevance labeling system performs across varying datasets.

US#2: As a researcher, I want to evaluate the impact different prompt templates have on the generated relevance labels.

US#3: As a researcher, I want to evaluate the impact different LLMs have on the generated relevance labels.

US#4: As a researcher, I want to evaluate the impact different model parameters have on the generated relevance labels.

US#5: As a researcher, I want to evaluate the impact different ensemble aggregation strategies have on the final relevance judgments.

US#6: As a researcher, I want to record all run-specific settings, including command-line flags, configuration files, version control state, and environment variables, in order to reproduce experimental results reliably.

US#7: As a donista employee, I want to evaluate the impact on the quality of the search results when adjusting the search engine.

US#8: As a donista employee, I want to track the performance of the search engine over time.

US#9: As a donista employee, I want to automate the process of evaluating the search engine.

Functional Requirements

The system is designed to fulfil the following core functionalities:

FR#1: The system must ingest different IR datasets and normalize them into a unified representation (derived from US#1).

FR#2: The system must automatically generate relevance judgments by prompting an ensemble of LLMs (derived from UC#1 and UC#2).

FR#3: The system must aggregate multiple model judgements to produce a final relevance label (derived from US#5).

FR#4: The system must compute evaluation metrics that quantify the quality and effectiveness of the generated relevance judgments (derived from US#2 to US#5 and US#7).

FR#5: The system must generate a run report that documents all run-specific settings (derived from US#6 and US#8).

FR#6: The system must validate configuration files prior to execution and provide clear, actionable error messages when configurations are invalid or incomplete (derived from UC#1 and UC#2).



Non-Functional Requirements

In addition to the core functionality, the system must meet the following quality and design constraints:

NFR#1: The system must provide clear and well-defined interfaces that allow easily exchanging datasets, prompt templates, model configurations, and provider integrations without requiring changes to the surrounding pipeline logic (derived from US#1 to US#4).

NFR#2: The system must ensure correctness across all pipeline stages through a comprehensive testing framework as executions may be long-running and late failures would be costly.

NFR#3: The system must persist individual relevance judgments immediately during execution to ensure that partial results progress is preserved in the event of interruptions or failures (derived from NFR#2).

NFR#4: The system must provide detailed and structured logging of key execution events, warnings, and errors to support post-run analysis and troubleshooting (derived from NFR#2).

NFR#5: The system must store all generated data in a structured and queryable format in order to support detailed data analysis (derived from US#1, US#4, US#5 and US#8).

NFR#6: The system must provide mechanisms for monitoring provider API usage and associated costs during execution. (derived from UC#1 and UC#2).

3.2 Software Architecture & Design

Ports & Adapters Architecture

A core requirement of the application is the interchangeability of components. Models, prompts, and model providers must be replaceable without requiring changes to the surrounding application logic (see US#1 to US#3 and US#5). The Ports & Adapters Architecture (also known as hexagonal architecture) was selected because it directly supports this requirement.

Cockburn [7] introduced the Ports & Adapters architecture in 2005. Its central principle is the strict separation of the application core from external dependencies, such as databases, user interfaces or other input and output. The application core interacts with its environment exclusively through abstract interfaces, referred to as *ports*. As a result, the core remains “blissfully ignorant” of concrete infrastructure implementation details.

Concrete implementations of these interfaces are referred to as *adapters*. Adapters encapsulate infrastructure-specific concerns and can be added, replaced, or modified without affecting the core application logic. This separation enables the system to remain flexible with interchangeable components.

As shown in Figure 3.1, each port represents a specific interaction point of the core, such as generating a prompt, invoking an LLM, or persisting the returned response. While the Ports & Adapters Architecture is traditionally applied to external infrastructure dependencies, this project extends the pattern to additional concerns such as prompt construction and parsing

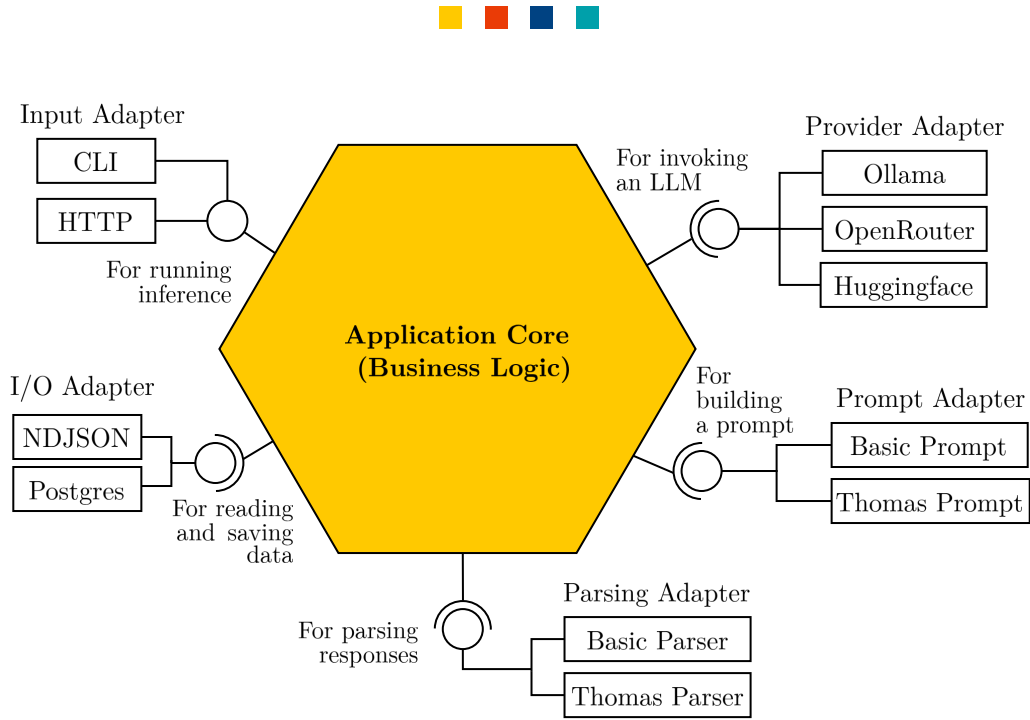


Figure 3.1: Ports & Adapters architecture, as exemplified in the INFER CLI. Some adapters are included only to demonstrate the flexibility of this architectural pattern and have not been implemented. Illustration adapted from Cockburn [8].

of LLM responses, as these aspects also benefit from the interchangeability requirements.

Cockburn and Paz also emphasize the importance of organizing software into layers with clearly defined dependency directions. These layers serve as a conceptual aid to ensure that the dependency inversion principle is respected: dependencies must always point inwards, and inner layers must not depend on outer layers. Paz, who collaborated closely with Cockburn, provides an example of this in the *bluezone* reference project [28].

Startup layer. The startup layer is responsible for assembling and launching the application. Within the Ports & Adapters architecture, it acts as a dependency configurator by selecting concrete adapters and injecting them into the application core based on configuration files.

Adapters layer. The adapters layer provides concrete implementations of infrastructure concerns such as persistence and communication with the provider. Each adapter implements a well defined interface defined by the application layer, ensuring loose coupling between business logic and technical implementation details.

Application layer. The application layer orchestrates the workflow, coordinating business logic with infrastructure adapters through port interfaces. Each application implements a main processing loop that iterates over domain entities and persists results through output adapters.

Domain layer. At the heart of the architecture lies the domain layer, which implements the system's core functionality and rules. This layer defines domain entities and provides functions which encapsulate business rules. The domain layer maintains strict independence from frameworks, infrastructure and other details to remain independently testable.



Pipeline Overview

The relevance labeling system is implemented as a modular four-stage pipeline. Each stage is represented by an independent entry point as illustrated in fig. 3.2. This design enables fine-grained control and composability, allowing stages to be executed independently or as part of a complete workflow. All pipeline stages are configurable through command-line flags, configuration files, and environment variables. Each stage produces a normalized output that serves as a data contract for subsequent stages.

The INGEST stage converts external datasets into a unified internal representation. While datasets may differ in schema, each must provide queries, documents, and a set of qrels that specify the relevance of each document with respect to a given query. Using a dataset-specific input adapter, this stage merges queries, documents, and qrels into standardized *JudgingSample* records. Together, these records constitute a *NormalizedDataset*, which serves as the input for the subsequent pipeline stage.

The INFER stage invokes an LLM to generate relevance judgements. For each sample in the input *NormalizedDataset*, a prompt-building adapter extracts the query and document and constructs instructions according to an adapter-specific prompt template. The LLM processes these instructions and returns a response expressing the estimated relevance of the document to the query. A response-parsing adapter parses the response and extracts the relevance judgement. In their entirety, these judgements constitute a *JudgedDataset*, which serves as the input for the subsequent pipeline stage.

In the AGGREGATE stage, relevance judgements from multiple LLMs are combined into a final *AggregatedJudgement* record. An aggregation adapter applies a specific aggregation strategy to merge multiple judgements into a single representation. To prevent invalid combinations, several validation steps are performed. Judgements are grouped and ordered by the sample they correspond to, ensuring that only compatible judgements are aggregated. In their entirety, the resulting aggregated judgements constitute the *AggregatedDataset*, which serves as the input for the subsequent pipeline stage.

Finally, the EVALUATE stage computes a set of evaluation metrics that quantify the agreement between

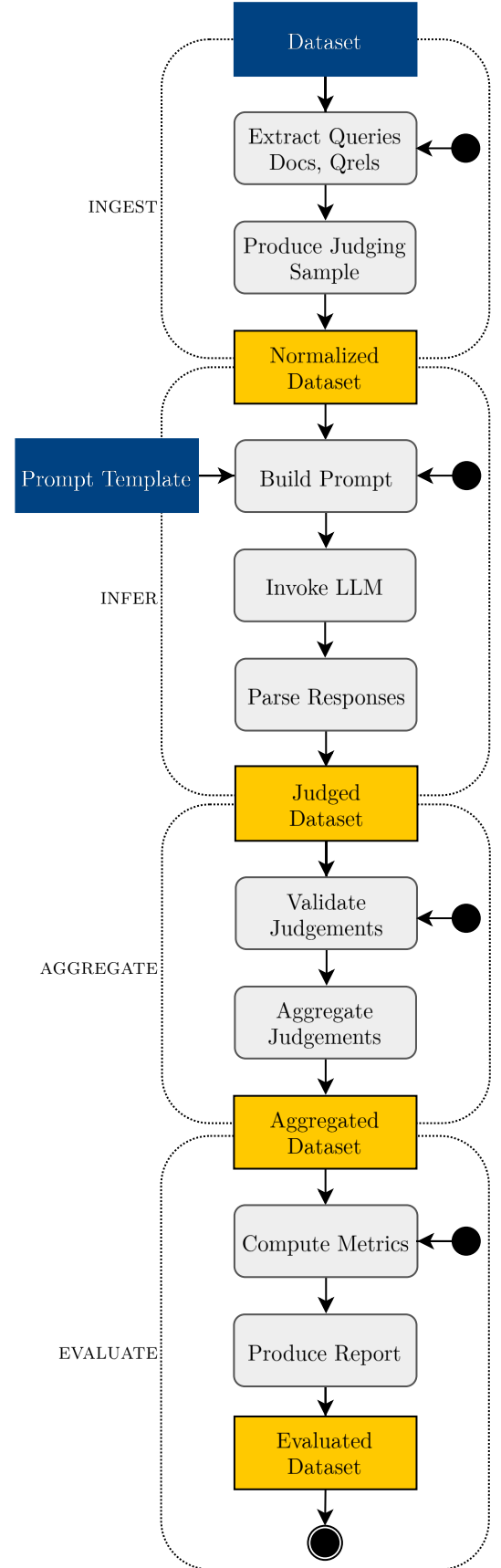


Figure 3.2: High-level object flow diagram of the pipeline.



LLM-based judgements and the reference judgements extracted during the INFER stage. This process is carried out by specialized metric-computation adapters. The stage produces a report that presents the computed metrics together with details on their computation, interpretation guidelines, and relevant metadata. In their entirety, these metrics constitute the *EvaluatedDataset*, which can be used for subsequent analysis.

Figures 3.3 to 3.5 illustrate the introduced entities and their relationships in the form of ER diagrams.

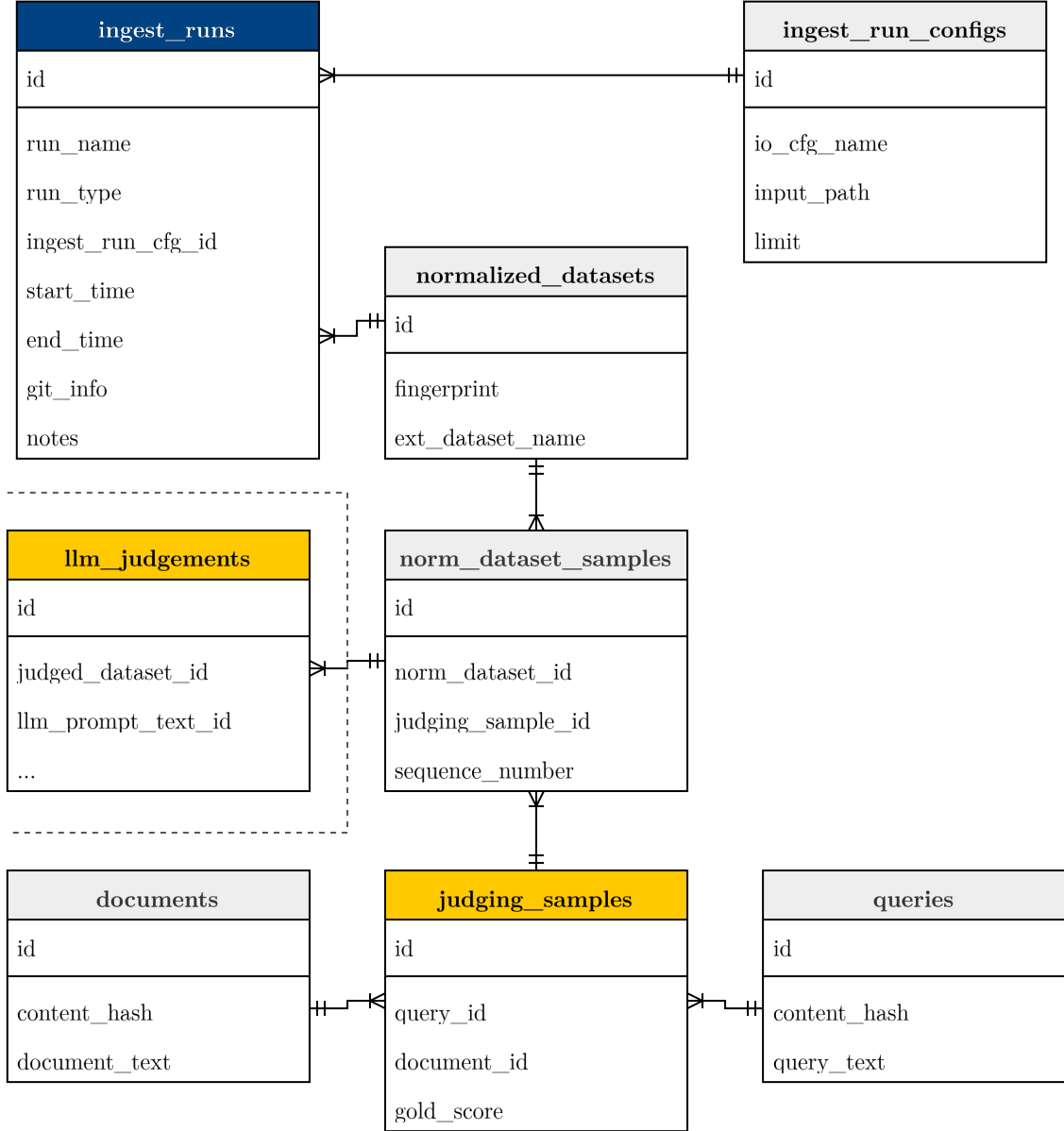


Figure 3.3: ER diagram of the INGEST pipeline in crow's foot notation. The dotted line indicates the boundary to the INFER pipeline.

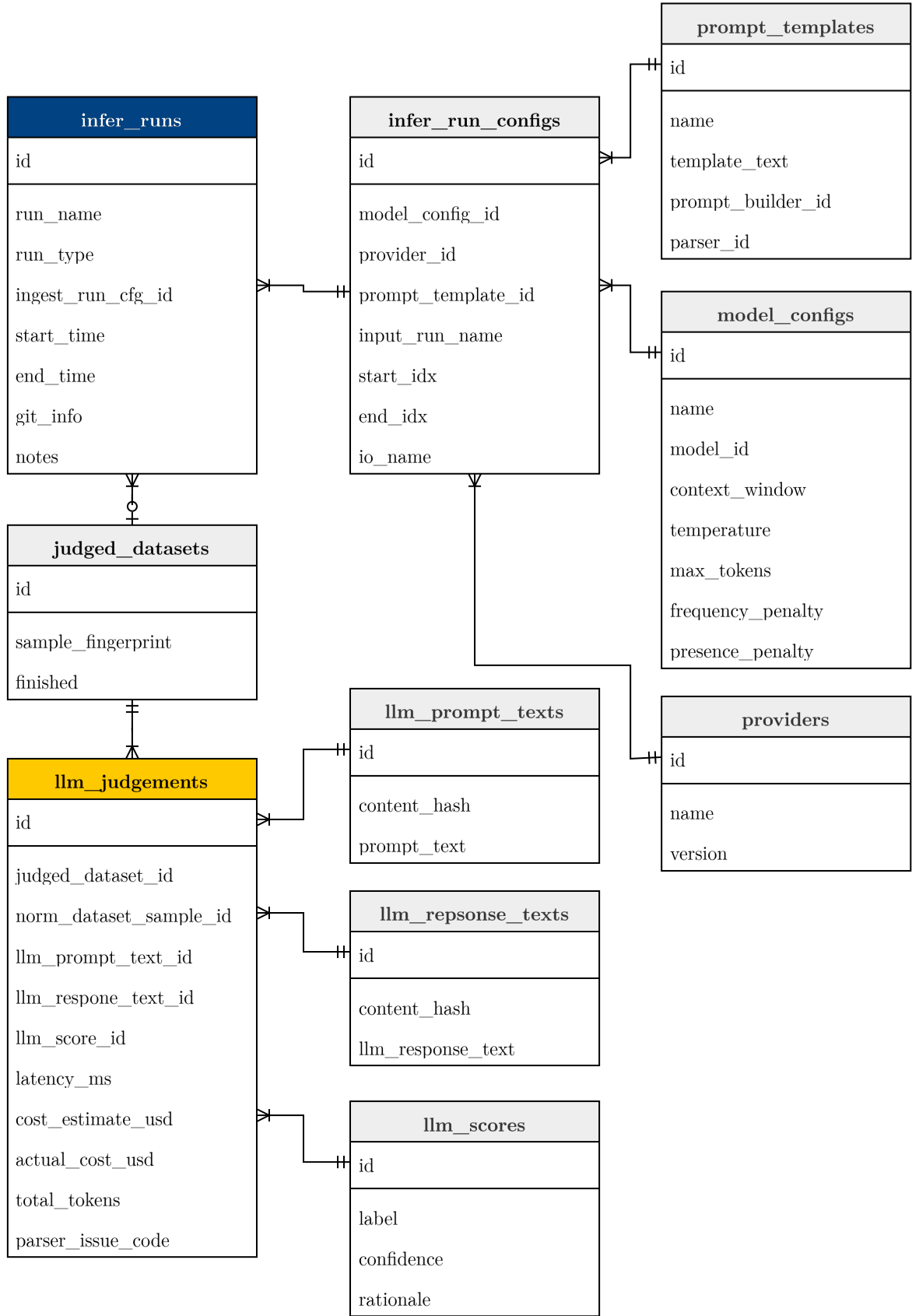


Figure 3.4: ER diagram of the INFER pipeline in crow's foot notation

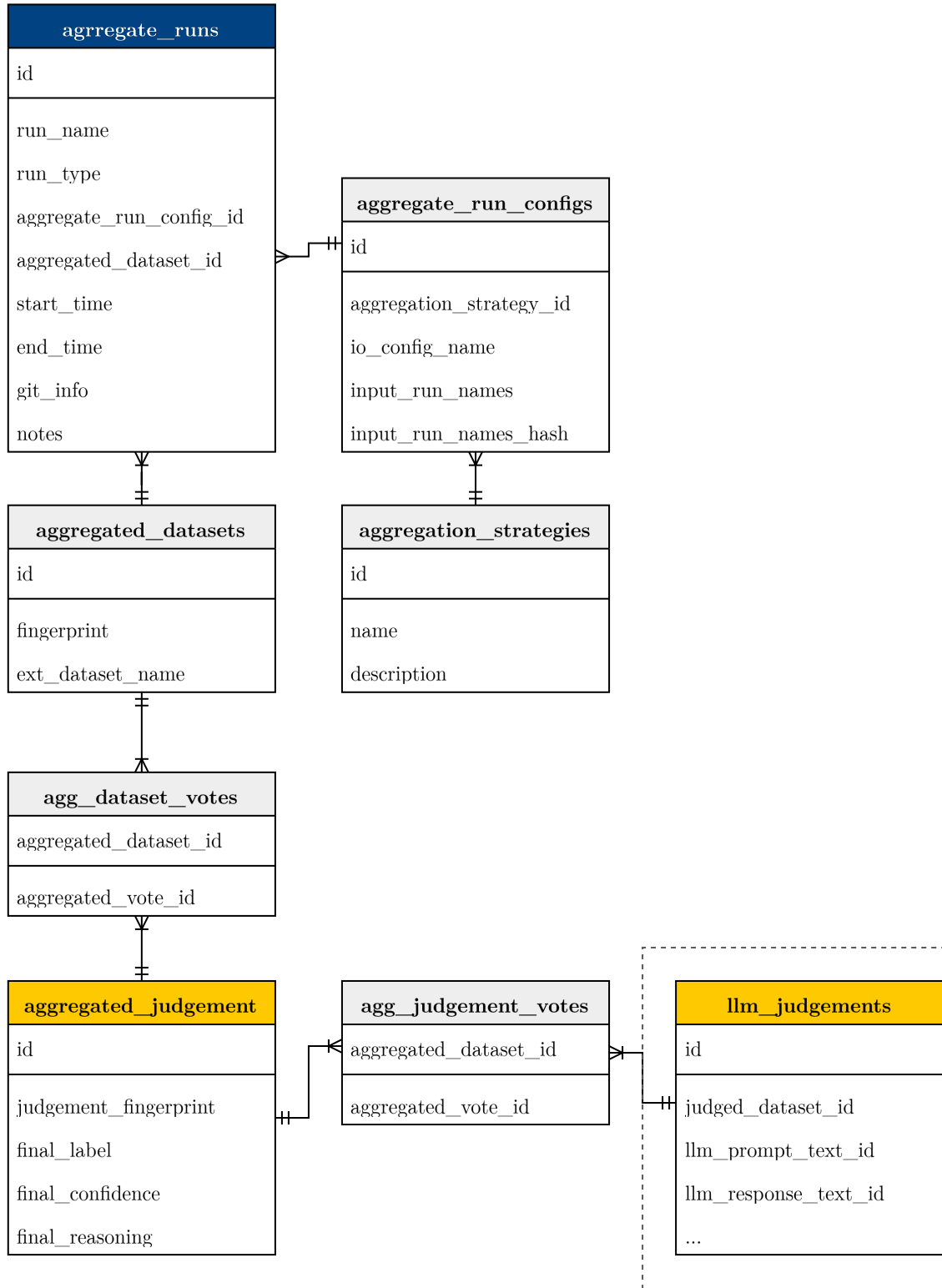


Figure 3.5: ER diagram of the AGGREGATE pipeline in crow's foot notation. The dotted line indicates the boundary to the INFER pipeline



3.3 Pipeline Implementation

Technology Choices

Python has become a standard in data science and machine learning research due to its extensive ecosystem of well-maintained libraries and strong community support [32]. Python further provides mature libraries for data processing, experimentation, and evaluation. To align this work with prevailing practices in the IR research community, Python was therefore adopted as the implementation language.

To support persistent storage and structured analysis of experimental results, a database was required. A relational database was chosen due to its strong support for normalized schemas, transactional consistency, and complex querying across related entities. PostgreSQL was selected because of its reliability, open-source ecosystem, and robust integration with Python through SQLAlchemy.

SQLAlchemy was used as the object-relational mapper due to its extensive documentation and long-term maintenance. An initial evaluation of SQLModel¹, a higher-level abstraction built on top of SQLAlchemy, was conducted, but it was ultimately excluded due to limited documentation.

To support both experimental workflows (UC#1) and automated execution in a deployment setting (UC#2), the system was designed to be containerized, which is a common requirement for consistent and reproducible system deployment. Docker was chosen as the containerization platform due to its broad adoption, strong community support, and seamless integration with the selected technology stack.

The system uses pytest as the testing framework, complemented by pytest-cov for coverage analysis. pytest was chosen due to its widespread adoption in the Python ecosystem and strong support for fixture-based testing. pytest-cov integrates seamlessly with pytest and provides lightweight, automated measurement of code coverage.

In the LLM Judge Challenge, Cohen’s κ is computed using `scikit-learn`, while Krippendorff’s α is computed using the `fast-krippendorff`² package. To ensure full comparability with the reported results, the same implementations are used in this work. All other evaluation metrics are also computed using `scikit-learn` unless stated otherwise. Token counts are computed using the `tiktoken` library.

Ingestion Stage

The INGEST pipeline is responsible for transforming external IR datasets into a unified internal representation. Table 3.1 summarizes the available options for configuring the execution of the INGEST command-line interface (CLI).

The workflow of the ingest pipeline consists of two primary steps. First, input data is read through a dataset-specific adapter. This adapter extracts queries, documents, and relevance judgements from the dataset it is responsible for and constructs corresponding *JudgingSample* entities. To prevent duplicate samples, a lightweight deduplication step is

¹<https://sqlmodel.tiangolo.com/>

²<https://github.com/pln-fing-udelar/fast-krippendorff>



Table 3.1: Command-line options for the INGEST CLI

Option	Required	Description
--input	yes	Path to data directory.
--io-cfg	yes	Specifies input and output adapters.
--limit	no	Process at most this many samples.
--run-name	no	Custom run name (auto-generates if not provided).
--official	no	Mark as official run.
--notes	no	Notes about this run.
--install-completion	no	Install tab autocomplete for the current shell.
--help	no	Show message with options.

applied that removes duplicate queries and documents based on their content. Finally, an output adapter persists the resulting structures, completing the ingest pipeline.

For each new dataset, only a corresponding input adapter must be implemented. This task is straightforward, as the input port precisely defines the required structures. This design allows multiple datasets to be integrated without modifying the INGEST application itself. Once an input adapter for a given dataset is implemented, the INGEST command can be executed without further modification. Consequently, experimentation across different IR datasets becomes straightforward. An example of such an adapter is `llm_judge_dataset_reader.py` in the thesis codebase.

For each execution of the INGEST pipeline, a structured run summary is generated and persisted under `/artifacts/runs/ingest`. The run summary captures high-level information about the execution. It includes the information listed in table 3.2. Execution logs produced during the run are stored in the same directory.

Table 3.2: Information contained in the ingest run summary.

Field	Description
Ingest Run configuration	Configuration used to execute the pipeline
Version control information	Git commit, branch, and repository state
Start time	Timestamp at which the run started
End time	Timestamp at which the run completed
Duration	Total execution time of the run
Sample count	Number of processed samples
Persistence summary	High-level summary of persisted artifacts

Inference Stage

The INFER pipeline is responsible for invoking an LLM to produce relevance judgements for a dataset that was normalized in the INGEST stage. Table 3.3 summarizes the options available to configure the execution of the INFER CLI.

All four pipelines follow the same directory structure. To avoid repetition, this structure is illustrated once using the INFER pipeline in fig. 3.6.



Table 3.3: Command-line options for the INFER CLI

Option	Required	Description
<code>--model-cfg</code>	yes	Model config name.
<code>--provider</code>	yes	Specifies provider adapter.
<code>--prompt-template</code>	yes	Specifies prompt builder and parser adapter.
<code>--io-cfg</code>	yes	Specifies input and output adapters.
<code>--input</code>	yes	Ingest run name to read samples from.
<code>--retry-cfg</code>	no	Retry config to use when requesting provider.
<code>--start-idx</code>	no	Start index into dataset.
<code>--end-idx</code>	no	End index into dataset.
<code>--run-name</code>	no	Custom run name (auto-generates if not provided).
<code>--official</code>	no	Mark as official run.
<code>--notes</code>	no	Notes about this run.
<code>--install-completion</code>	no	Install tab autocomplete for current shell.
<code>--help</code>	no	Show message with options.

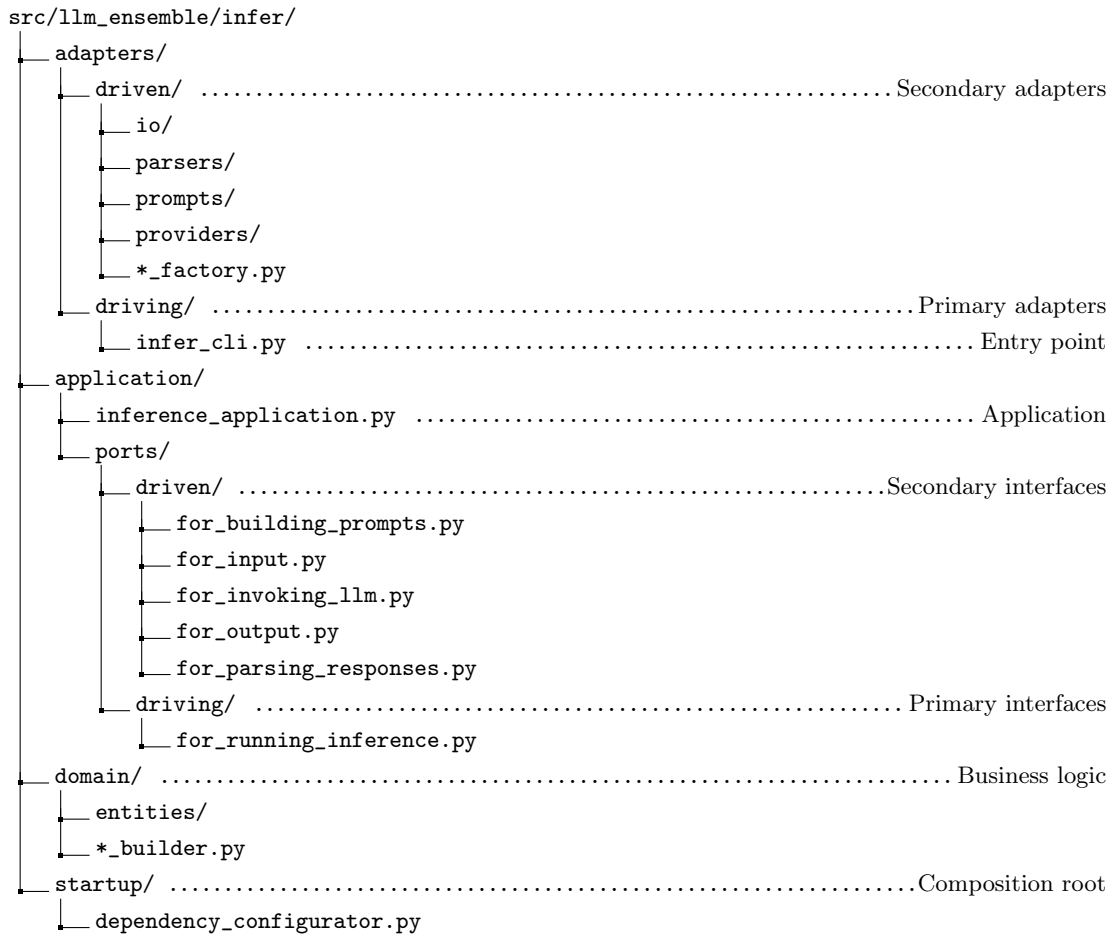


Figure 3.6: Directory structure of the INFER module. The folder structure divides the module into the adapter, application, domain, and startup layer.



Listing 3.1 shows the core inference loop, adapted from `inference_application.py`. The application begins by loading a *NormalizedDataset* into memory using an input adapter. The dataset is processed sequentially on a per-sample basis. For each sample, the prompt-building adapter extracts the associated query and document and constructs a prompt according to its configured template. The prompt is then submitted to the LLM through an inference adapter, which applies the retry strategy specified via the CLI when issuing requests. This strategy is based on exponential backoff with configurable delays, timeouts, and retry limits, and aborts the run if these limits are exceeded. The resulting response is subsequently parsed by a response-parsing adapter, which extracts the relevance label.

The prompt-building adapter and the response-parsing adapter are intentionally tightly coupled. As shown in table 3.3, users only need to specify the prompt template to be used. This is because each prompt template enforces a specific output structure, which the corresponding parser relies on to reliably extract relevance labels.

The response-parsing adapters implement a staged extraction procedure to robustly handle non-conforming LLM outputs. Rather than assuming that the response strictly follows the requested format, the parser applies a cascade of increasingly permissive strategies. The first strategy that yields a valid label terminates the cascade. To support analysis of LLM behaviour, the parser optionally returns parsing issue codes alongside the extracted score. These codes represent specific failure modes, such as fully failed extraction, missing required fields, invalid field values, and non-standard response formats.

Inference execution follows the context manager pattern (line 8 in listing 3.1), ensuring that all acquired resources are released, even in the presence of exceptions. Run-level configuration and metadata are persisted in the opening statement of the context manager. This design prevents long-running INFER jobs from failing late due to persistence errors.

During processing, each judgement is persisted immediately after it is produced. Since LLM invocations represent the most costly part of the pipeline, this approach avoids data loss in the event of unexpected failures. The construction of individual judgements follows a builder pattern, allowing information produced incrementally by different adapters to be assembled into a complete judgement object.

For each execution of the INFER pipeline, a structured run summary is generated and persisted under `/artifacts/runs/infer`. The run summary captures high-level information about the execution. It includes the information listed in table 3.4. Execution logs produced during the run are stored in the same directory.

Table 3.4: Information contained in the infer run summary.

Field	Description
Infer run configuration	Model, prompt template, retry, and execution settings
Version control information	Git commit, branch, and repository state
Start time	Timestamp at which the run started
End time	Timestamp at which the run completed
Duration	Total execution time of the run
Judgement statistics	Count of judgements, failed parses, and vote breakdown
Performance metrics	Token usage, cost information and API issues
Persistence summary	High-level summary of persisted artifacts



```
1 start_time = datetime.now()
2
3 # Read NormalizedDataset
4 normalized_dataset = self.input_port.read(input_run_name)
5 samples_to_process = normalized_dataset.samples[start_idx:end_idx]
6
7 # Open writer for streaming output
8 with self.output_port.open(infer_run) as writer:
9
10     # Process each sample in slice
11     for sample in samples_to_process:
12
13         # Initialize builder for this sample
14         builder = LLMJudgementBuilder(sample)
15
16         # Step 1: Build prompt
17         prompt_text = self.prompt_builder.build_prompt(sample)
18         builder.with_prompt(prompt_text)
19
20         # Step 2: Run inference
21         response = self.llm_provider.infer(prompt_text)
22         builder.with_llm_response(response)
23
24         # Step 3: Parse response
25         llm_score, parser_issue = self.response_parser.parse(
26             response)
27         builder.with_score(llm_score, parser_issue)
28
29         # Write judgement immediately to disk (fault tolerance)
30         judgement = builder.build()
31         writer.write_one(judgement)
32
33 end_time = datetime.now()
34
35 # Build summary for run report
36 run_summary = self._build_run_summary(
37     start_time,
38     end_time,
39     llm_judgements
40 )
41
42 # Write run summary
43 run_summary_path = persist_run_summary(run_summary, self.run_dir)
44 return run_summary_path
```

Listing 3.1: Core inference loop of the application layer coordinating prompt construction, LLM invocation, parsing, and incremental persistence. Logging and other small details have been removed for brevity.



Aggregation Stage

The AGGREGATE pipeline combines relevance judgements produced by multiple INFER runs into a single *AggregatedJudgement*. The set of input judgements is determined by the infer run names provided to the application. Table 3.5 summarizes the options available to configure the execution of the AGGREGATE CLI.

Table 3.5: Command-line options for the AGGREGATE CLI

Option	Required	Description
--agg-strategy	yes	Specifies aggregation adapter.
--io-cfg	yes	Specifies input and output adapters.
--input	yes	Infer run names to read judgements from. Can specify multiple.
--run-name	no	Custom run name (auto-generates if not provided).
--official	no	Mark as official run.
--notes	no	Notes about this run.
--install-completion	no	Install tab autocomplete for the current shell.
--help	no	Show message with options.

An input adapter first resolves the specified infer run identifiers and loads the corresponding *LLMJudgement* entities. These judgements are passed to a single aggregation adapter selected by the application. The aggregation adapter encapsulates the complete aggregation logic.

Internally, the aggregation adapter groups judgements by their associated query-document pair. Within each group, judgements from different infer runs are aggregated according to the selected strategy, yielding a single aggregated vote per sample.

In addition to the final relevance label, aggregation adapters attach auxiliary metadata to each *AggregatedJudgement*. This metadata includes confidence estimates derived from vote distributions, explicit tracking of tie situations, and a concise reasoning string that documents how the final decision was obtained.

A central concern of the AGGREGATE pipeline is validation. A user may inadvertently specify INFER runs that are not mutually compatible. As a result, the pipeline must ensure that only judgements referring to the same *JudgingSample* are combined.

To ensure that only compatible judgements are combined, a deterministic fingerprint is computed for each *JudgedDataset*. The fingerprint is derived by hashing the sorted identifiers of the associated *JudgingSample* entities and uniquely represents the set of query-document pairs covered by a run. Only *JudgedDataset* instances with identical fingerprints are eligible for aggregation.

This validation prevents the combination of judgements that originate from different query-document pairs. Without this constraint, aggregation could accidentally merge relevance labels referring to unrelated samples.



Table 3.6: Example of incompatible judgements that must not be aggregated

LLMJudgement	Query	Document	Score
A	What is Python?	A programming language.	3
B	How tall is Mount Everest?	Mount Everest has an elevation of 8,848 meters above sea level.	1

Evaluation Stage

The EVALUATE pipeline computes evaluation metrics by comparing model judgements against ground truth labels. Table 3.7 summarizes the options available to configure the execution of the EVALUATE CLI.

The evaluation pipeline is designed to operate on the output of either INFER or AGGREGATE runs. This flexibility allows both individual model runs and aggregated ensemble outputs to be evaluated using the same evaluation infrastructure. The concrete input format is selected dynamically through the configured I/O adapter, based on the provided I/O configuration.

Once the input is resolved, the application provides each metric computation adapter two aligned collections: a list of relevance judgements that was produced in the earlier pipeline stages, and the corresponding ground truth relevance labels. These two lists form the basis for computing evaluation metrics, such as inter-rater reliability (a measure of consistency between independent raters) measures.

Each evaluation metric is implemented as a dedicated metric adapter. Metric adapters define not only the computation of the metric itself, but also associated metadata, including a description of the metric, its valid value range, and interpretation guidelines. This information is used to enrich the evaluation output with automated, human-readable interpretations of the results.

Upon completion, the evaluation stage produces a structured run report that consolidates the computed metrics and their associated metadata. At present, this report is emitted as a simple JSON artifact which proved to be sufficient enough. Examples can be found in the codebase under `/artifacts/runs/evaluate`.

Table 3.7: Command-line options for the EVALUATE CLI

Option	Required	Description
<code>--input</code>	yes	Input run name.
<code>--metric</code>	yes	Specifies metric adapters.
<code>--io-cfg</code>	yes	Specifies input and output adapters.
<code>--run-name</code>	no	Custom run name (auto-generates if not provided).
<code>--official</code>	no	Mark as official run.
<code>--notes</code>	no	Notes about this run.
<code>--install-completion</code>	no	Install tab autocomplete for current shell.
<code>--help</code>	no	Show message with options.



3.4 Testing

The Ports & Adapters architecture enforces strict segregation between the application core and business rules from outside dependencies. This facilitates a comprehensive testing infrastructure which is a key advantage of this architecture and directly supports NFR#2. Since interactions with the outside world occur only through defined ports, adapters such as databases or LLM providers can be replaced with test doubles (depicted in fig. 3.7). As a result, business logic can be tested reliably and in isolation from infrastructure. Listing 3.2 demonstrates this well.

Tests are classified using pytest markers defined in `pyproject.toml`. The testing philosophy at donista prioritizes integration tests and supplements them with unit tests for particularly critical functionality. To ensure consistency and a smooth transition, this project adopts the same testing approach.

Most tests are marked with one of the two:

- `@pytest.mark.unit`: Fast, fully isolated tests without I/O dependencies. These tests validate pure business logic and domain entities independently of the infrastructure layer.
- `@pytest.mark.integration`: Tests that involve the file system, database adapters, or other infrastructure components. These ensure that adapters correctly implement the defined port interfaces.

Additional markers, such as those for tests requiring API access or other external resources, are available but have proven to be of limited practical value. In total, 74 tests were implemented for the four pipeline stages, achieving 89% code coverage as measured by `pytest-cov`. Tests are separated from source code and located under `/tests/`.

A particular focus was placed on testing the EVALUATE pipeline. Although the concrete implementation of evaluation metrics is delegated to established libraries such as `scikit-learn`, ambiguities remain in how these APIs are configured and applied. For example, metrics such as Krippendorff's α can be computed under different assumptions, including ordinal or interval-scaled data. To ensure that metrics are computed in a manner consistent with the LLM Judge Challenge, the evaluation pipeline was validated against the official results of the challenge. For this purpose, the tests recompute metrics for eight submitted runs and compared to the values reported by the challenge organizers. A small numerical tolerance of 0.001 was allowed to account for minor floating-point deviations. This validation confirms that the evaluation pipeline reproduces the published results with sufficient accuracy and can therefore be relied upon in subsequent experiments.

Another area of particular importance that lends itself well to unit testing is the response-parsing logic. The response parsers are exercised using a wide range of intentionally malformed or non-conforming model outputs to verify their behaviour. As described earlier, the INFER pipeline applies a staged extraction procedure in which multiple parsing strategies are attempted in order of decreasing strictness.

Unit tests validate that the parser correctly applies this cascade, terminates at the first successful extraction, and reliably reports parsing issue codes when irregularities are detected. These issue codes are essential for deriving meaningful statistics about model behaviour. Ensuring the correctness of this behaviour through unit tests is therefore critical for both the robustness of inference and the validity of subsequent analytical insights.

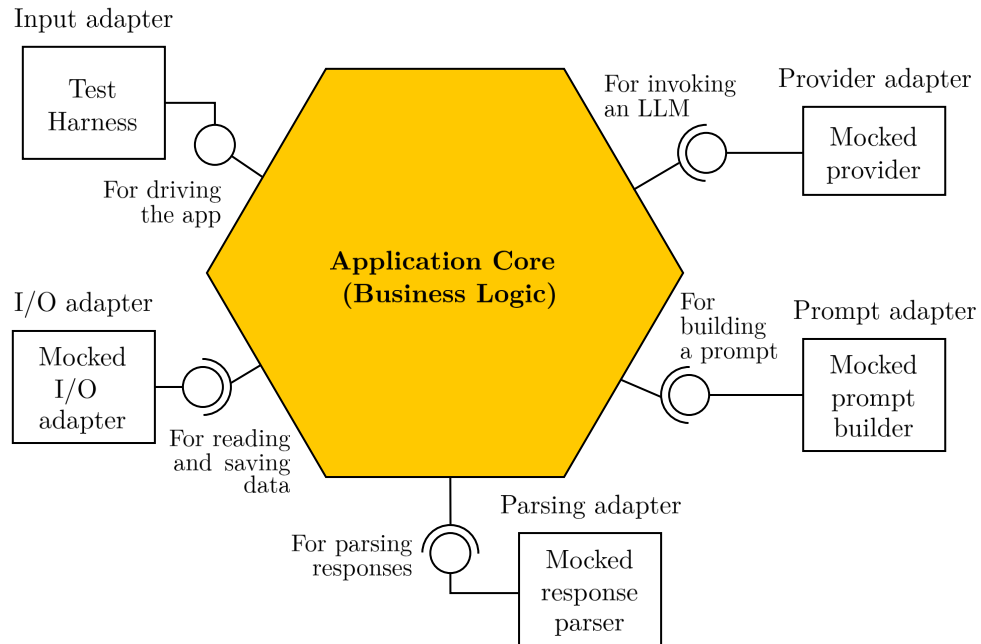


Figure 3.7: Illustration of the testing architecture. The application core is surrounded by mocked adapters that simulate external dependencies such as input, I/O, provider, prompt, and parsing components. Illustration adapted from Cockburn [8].

```
1
2 @pytest.mark.integration
3 def test_inference_pipeline_slice(
4     sample_data_five: NormalizedDataset,
5     temp_run_dir: Path
6 ):
7     # Arrange: Inject real adapters + mock infrastructure
8     app = InferenceApplication(
9         input_port=MockInputAdapter(sample_data_five),
10        output_port=MockOutputAdapter(),
11        prompt_builder=ThomasAdvancedPromptBuilder(),
12        provider=MockProvider(mock_response='{"M":2,"T":1,"O":1}'),
13        response_parser=ThomasAdvancedParser(),
14        run_dir=temp_run_dir,
15        run_name="integration-test-run"
16    )
17
18    # Act: process samples at indices 2 and 3
19    summary = app.run_inference(
20        input_run_name="test-ingest-run",
21        start_idx=2,
22        end_idx=4,
23        official=False,
24        notes="Integration_test_with_real_adapters"
25    )
26
27    # Assert: Only sliced samples were processed
28    assert summary.judgements.total_count == 2
29    assert len(output_adapter.written_judgements) == 2
30
```



```
31 # Assert: Correct samples from dataset were processed
32 first_judgement = output_adapter.written_judgements[0]
33 second_judgement = output_adapter.written_judgements[1]
34 assert first_judgement.dataset_sample == sample_dataset_five.
    samples[2]
35 assert first_judgement.dataset_sample.sequence_number == 2
36 assert second_judgement.dataset_sample == sample_dataset_five.
    samples[3]
37 assert second_judgement.dataset_sample.sequence_number == 3
38
39 # Assert: parser successfully extracted scores from samples
40 assert summary.judgements.failed_parsing_count == 0
41 assert first_judgement.llm_score is not None
42 assert first_judgement.llm_score.label == RelevanceScore.
    RELEVANT
43 assert second_judgement.llm_score is not None
44 assert second_judgement.llm_score.label == RelevanceScore.
    RELEVANT
45
46 # Assert: Application calculated metrics
47 assert summary.performance.latency.total_ms is not None
48 assert summary.performance.latency.avg_ms is not None
49 assert summary.judgements.total_count is not None
50 assert summary.judgements.failed_parsing_count is not None
51 assert summary.performance.cost.total_usd is not None
52 assert summary.performance.tokens.total is not None
53 assert summary.performance.tokens.total_prompt is not None
54 assert summary.performance.tokens.total_completion is not None
```

Listing 3.2: Example test for the inference pipeline. Core application components, including the prompt builder and response parser, are instantiated using their real implementations. External dependencies such as the LLM provider and file system interactions are replaced with test doubles. This test is able to cover the INFER pipeline almost end to end and verifies the core workflow runs as intended.



Chapter 4

Methodology

This chapter describes the methodology used to evaluate the proposed relevance labelling system. It introduces the datasets underlying the LLM Judge Challenge and details how withheld evaluation labels were reconstructed and verified. It then details the selection of the reference model, ensemble members, and the prompt used throughout the experiments. Finally, it specifies the experimental design and the evaluation metrics used to quantify agreement with the ground truth human labels.

4.1 Underlying Dataset

The system developed in this thesis was evaluated against the dataset provided by the LLM Judge Challenge. This dataset is derived from the TREC 2023 Deep Learning Track. The TREC Deep Learning Track itself is based on the Microsoft Machine Reading Comprehension (MS Marco) dataset, which serves as the underlying corpus.

Table 4.1: LLM Judge Challenge Dataset Statistics (Test Split). Only unique documents that were rated are included.

Metric	Value
Total queries	25
Total documents	3,909
Total qrels	4,423

Table 4.2: Label Distribution for the LLM Judge Challenge

Label	Meaning	Count	Percentage
3	Perfectly relevant	377	8.52%
2	Highly relevant	808	18.27%
1	Related	1,233	27.88%
0	Irrelevant	2,005	45.33%



History

This section briefly reviews the history of previously mentioned datasets, in order to understand their relationship more closely. Knowledge of their relationship proved to be crucial in an effort to recover publicly unavailable relevance labels.

In 2016, Nguyen et al. [25] from Microsoft released the MS Marco dataset featuring 100,000 anonymized search queries sent to the Bing search engine. In 2018, Bajaj et al. [2] introduced MS Marco v2, containing more than 1 million anonymized search queries with more than 8 million related passages extracted from approximately 3 million web documents. Due to its large and diverse variety of real-world search examples, the dataset has been widely used for IR benchmarking [38, 17, 31].

In 2019, TREC started incorporating the MS Marco dataset, hosting a number of competitions with tasks such as document and passage retrieval. Moreover, the NIST, host of the TREC, reevaluated the binary relevance labels from MS Marco with the help of human assessors. They introduced a four-point relevance scale, which allows for more comprehensive metrics to be applied [9].

Table 4.3: TREC relevance scale for passages.

Label	Description
3	Perfectly relevant: passage is dedicated to the query and contains the exact answer.
2	Highly relevant: passage contains an answer, but it may be unclear or obscured by extraneous information.
1	Related: passage is related to the query but does not provide an answer.
0	Irrelevant: passage has nothing to do with the query.

In early 2025, the LLM Judge Challenge [31] concluded. 42 teams submitted runs that used LLMs to generate relevance judgements for a subset of the TREC 2023 Deep Learning Track (see section 2.3). The dataset was split into a development set and a held-out evaluation set used for the official challenge results. The organizers did not release the relevance judgements used for the official evaluation. Only the relevance judgements for the development set were made publicly available.

This limits direct comparison with the challenge submissions. Evaluation on the development set alone reduces comparability, as relevance labels would correspond to different query-document pairs. However, the challenge’s reliance on the TREC 2023 Deep Learning Track enabled recovery of the relevance judgements for the official evaluation split. The recovery process is described in the following section.

Recovering Relevance Labels

The LLM Judge Challenge did not release the relevance labels for its official evaluation split. We therefore reconstructed these labels using a simple matching and verification pipeline. All scripts implementing the step-by-step recovery process are available in the project codebase under `/scripts/trec_qrels_recovery`. The recovered relevance labels are stored in `/datasets/llm_judge_challenge_qrels_recovered`.



The recovery was possible because the challenge organizers provided mapping files that link anonymized identifiers used in the LLM Judge Challenge to the original MS Marco identifiers. By combining these mappings with publicly available TREC resources, we were able to trace back each challenge query to its original relevance judgments.

First, we used the provided mapping files to link the anonymized LLM Judge Challenge identifiers back to their original MS Marco identifiers. We then matched these identifiers against the official TREC 2023 Deep Learning Track queries obtained via `ir_datasets`. This step allowed us to identify the corresponding TREC queries for all challenge instances.

Second, we downloaded the official TREC 2023 passage qrels from NIST [26]. These qrels contain graded relevance judgments for the full track. We filtered this file to retain only the 50 queries used in the LLM Judge Challenge, yielding a candidate set of recovered relevance labels.

Finally, we verified that the recovered qrels are identical to those used in the LLM Judge Challenge. Since the organizers released all participant submissions together with their generated relevance labels [29], we were able to do this by recomputing the agreement metrics. If the recomputed values match the metrics published in the LLM Judge Challenge, the recovered labels must be identical to the original gold labels.

We performed this verification for the first listed challenge submission, `NISTRetrieval-instruct0` [31]. The recomputed Cohen’s κ and Krippendorff’s α match the reported values to four decimal places. This confirms that the recovered relevance labels are the same as those used in the LLM Judge Challenge.

4.2 Model Selection

To assess the effectiveness of an LLM ensemble, a single-model baseline is required. This baseline provides a reference point for interpreting ensemble performance under identical evaluation conditions. As representatives of strong single-model performance, we selected state-of-the-art models based on their relative standing in the LM Arena text-based evaluation leaderboard [20].

Choosing the Reference Model

The technical lead at donista had defined an upper bound of 20 USD on the cost incurred per run. Consequently, models whose per-run cost exceeded this limit were excluded from further consideration, as they do not satisfy the operational constraints.

To estimate expected evaluation costs, we conducted a small-scale smoke test using 100 samples for each model. Model selection was further constrained by availability through the OpenRouter API at the time of evaluation. The resulting measurements were recorded under `/artifacts/runs/infer/official` in the project codebase. The observed costs were extrapolated to the full evaluation set of 4,423 samples, enabling comparison of performance and cost under a common workload. This extrapolation assumes that a future dataset at donista will be of comparable size.

Based on the dataset characteristics shown in table 4.4, GPT 5.1 was selected as the single-model reference baseline, as it offers a strong performance–cost trade-off and satisfies the defined cost constraints.



Table 4.4: Single-model baselines. Inference costs are reported in USD. Per-sample costs are estimated from a 100-sample smoke test and extrapolated to the full evaluation set of 4,423 judged query-document pairs (see table 4.1).

Model	Cost (100 samples)	Estimated total cost	LM Arena rank
Claude Opus 4.5	1.8039	80.03	5
GPT 5.2	0.6479	28.66	14
GPT 5.1	0.4887	21.62	20

Choosing the Ensemble Models

The motivation for using an LLM ensemble instead of a single model rests on two main considerations. These considerations guided the selection of ensemble members.

First, ensembles can be cost-efficient. Large state-of-the-art models are often expensive per inference. In contrast, several smaller models can be engineered for speed and low cost. Individually, these models tend to perform worse. However, their aggregated output may rival or exceed the performance of a single stronger model at lower overall cost [13].

Second, ensembles can benefit from a *wisdom of the crowd* effect. This effect can be attributed to two factors. The first is vote aggregation: a larger number of independent judgments increases confidence in the final decision. The second is diversity: models with different architectures, training data, or optimization objectives may exhibit different failure modes. In such cases, aggregation can reduce systematic errors by compensating for individual weaknesses [13].

This work does not attempt to disentangle the relative importance of vote count versus model diversity. For the intended operation at donista, this distinction is of limited practical value. As long as the ensemble produces reliable judgments at low cost, the underlying cause of the improvement is secondary. We therefore assume that model diversity plays a role, without claiming strong empirical evidence in support of this assumption. To remain conservative, we deliberately select a diverse set of models.

Both diversity and cost must therefore be considered when selecting ensemble members. Accordingly, we began by filtering candidate models based on a set of operational constraints. Table 4.5 summarizes this filtering process.

Cost constraint. Free models were excluded, as they are not meaningful for the cost analysis conducted later. To ensure that the ensemble offers a clear cost advantage over the reference model, we imposed a pricing constraint. Since the ensemble is evaluated with up to five models, the average cost per model must not exceed one fifth of the reference model’s cost. In addition, the ensemble is required to cost at most half as much as the reference model in order to justify the additional development and organizational overhead. Taken together, these constraints imply that the average cost per model must not exceed one tenth of the reference model’s cost. Model cost is estimated as the arithmetic mean of the input and output token prices reported by OpenRouter. While this estimate does not account for tokenizer-specific effects, it is sufficient for relative cost comparison.

Parameter count constraint. In addition to cost, we applied a constraint on model parameter count. Since donista is considering self-hosting LLMs, response latency and hardware requirements are critical. Based on prior experience hosting models locally, models



exceeding approximately 10 billion parameters require substantially more capable hardware to maintain acceptable response times. We therefore excluded models with more than 10 billion parameters, as well as models for which no reliable parameter count could be determined.

Context window constraint. The context window constraint is derived from the prompt template (see listing 4.1) and the characteristics of the LLM Judge Challenge dataset. A small script based on the `tiktoken` library was used to estimate token counts. It computes the maximum expected token length by combining the prompt template with the largest query–document pair in the dataset. This analysis yielded a maximum expected length of 93,148 tokens. We therefore excluded all models whose context window is smaller than this value.

After applying all filters, 15 candidate models remained.

Table 4.5: Model filtering steps applied to the OpenRouter catalog.

Filtering step	Remaining	Excluded
Initial catalog	347	–
Remove free models	315	32
Apply cost constraint	144	171
Apply parameter count constraint	39	105
Apply context window constraint	15	24

Table 4.6 summarizes the remaining candidate models considered for ensemble construction. The table reports a subset of the collected attributes; additional metadata and extended comparisons are available in the source file at `/artifacts/tables/model-analysis`. Use case descriptions were taken from provider documentation. Information on training data is included where publicly available, but is incomplete for some models.

During model selection, preference was given to a low parameter count while maintaining sufficient diversity. A lower parameter count was favored because it makes self-hosting more feasible. Two models, `granite-4.0-h-micro` by IBM and `nemotron-nano-9b-v2` by NVIDIA, were excluded due to unreliable endpoint behavior during evaluation.

The initial model selection for the ensemble consists of the following five models:

- `meta-llama/llama-3.2-3b-instruct`
- `mistralai/ministral-3b-2512`
- `google/gemma-3-4b-it`
- `microsoft/phi-4-multimodal-instruct`
- `bytedance/ui-tars-1.5-7b`

This selection was informed but ultimately heuristic. It aims to balance model diversity, inference cost, and the practical feasibility of self-hosting.

Table 4.6: Ensemble candidates that satisfied the operational constraints. Sorted by parameter size.

Model	Provider	Loc	Size	Use Case	Input	Training Data
granite-4.0-h-micro	IBM	US	3B	context tool calling	Text	open-source instruction datasets
llama-3.2-3b-instruct	Meta	US	3B	multilingual dialogue, agentic retrieval, summarization	Text	web
ministral-3b-2512	Mistralai	FR	3B	image captioning, classification, real-time translation, data extraction, content generation	Text, Image	
gemma-3-4b-it	Google	US	4B	text generation, chatbots, summarization, image data extraction, NLP, knowledge exploration	Text, Image	web, code, math, images
phi-4-multimodal-instruct	Microsoft	US	6B	low-latency, edge and mobile deployments	Text, Image, Audio	web, synthetic data, human-labeled chats, speeches
ui-tars-1.5-7b	Bytedance	CN	7B	GUI environments, desktop interfaces, web browsers, mobile systems, games	Text, Image	GUI screenshots, interaction traces, tutorials, synthetic data
command-r7b-12-2024	Cohere	CA	7B	high throughput, latency-sensitive chatbots, code assistants, RAG, tool use	Text	web, code, technical literature
deepseek-r1-0528-qwen3-8b	Deepseek	CN	8B			
llama-guard-3-8b	Meta	US	8B	content safety classification	Text	hh-rlhf dataset, human-ai conversations, synthetic data
ministral-8b-2512	Mistralai	FR	8B	image and document description, translation, content generation	Text, Image	
qwen3-8b	Alibaba	CN	8B	reasoning-heavy tasks, efficient dialogue, creative writing, role-playing, dialogues	Text	web, PDF-like documents, synthetic data: textbooks, code
qwen3-vl-8b-instruct	Alibaba	CN	8B	document parsing, visual question answering, spatial reasoning, and GUI control	Text, Image, Video	web, PDF-like documents, synthetic data: textbooks, code
nemotron-nano-9b-v2	Nvidia	US	9B	general purpose reasoning, code, chat	Text	





4.3 Prompt Selection

The influence of prompting on relevance judgments, commonly referred to as *prompt sensitivity*, is difficult to characterize and exhibits high variability across settings [37, 40, 17]. Small changes in prompt formulation can lead to substantial differences in model outputs and relevance labels. As a result, prompt choice has a strong impact on evaluation outcomes.

Thomas et al. [37] conducted a detailed study on prompt sensitivity. Several recent studies adopt the prompt developed by them [38, 30, 31]. To ensure comparability with prior work, we follow this prompt formulation as closely as possible.

However, adopting this prompt introduces a mismatch in relevance labeling. Thomas et al. [37] use a three-point relevance scale, whereas the LLM Judge Challenge adopts a four-point scale (see table 4.3). This discrepancy leaves two options. One option is to remap the four-point judgments to a three-point scale. This would reduce comparability by discarding information present in the original labels. The alternative is to adapt the prompt to the four-point scale used by the LLM Judge Challenge, at the cost of minor deviations from prior prompt formulations.

We adopted the latter approach. The objective of this thesis is not to study prompt features, but to evaluate the effectiveness of LLM ensembles. Maintaining direct comparability with the LLM Judge Challenge results therefore took precedence.

Thomas et al. [37] evaluate combinations of prompt features on the TREC 2004 Robust dataset. They identify five feature categories: (i) role, (ii) description, (iii) narrative, (iv) aspects, and (v) ensemble prompting. Their best-performing configuration combines descriptive, narrative, and aspect-based instructions. However, the LLM Judge Challenge does not provide description or narrative fields. Under these constraints, the aspect-only variant achieves the strongest average performance. We therefore adopted this prompt variant, shown in listing 4.1.

Listing 4.1: Prompt adapted from Thomas et al. [37] to the TREC relevance scale.

```
Given a query and a web page, you must provide a score on an integer scale of 0
to 3 with the following meanings:

3 = Perfectly relevant: The passage is dedicated to the query and contains the
exact answer.
2 = Highly relevant: The passage has some answers for the query, but the answer
may be a bit unclear, or hidden amongst extraneous information
1 = Related: The passage seems related to the query but does not answer it.
0 = Irrelevant: The passage has nothing to do with the query.

Assume that you are writing a report on the subject of the topic.
If the passage directly and clearly answers the query, mark it 3.
If it is primarily about the topic but the answer is partial or indirect, mark it
2.
If it is only loosely related but might provide minor context, mark it 1.
Otherwise, mark it 0.

Query
A person has typed [{query}] into a search engine.

Result
Consider the following web page.
```



```
-BEGIN WEB PAGE CONTENT-  
{document}  
-END WEB PAGE CONTENT-
```

Instructions

Split this problem into steps:

Consider the underlying intent of the search.

Measure how well the content matches a likely intent of the query (M).

Measure how trustworthy the web page is (T).

Consider the aspects above and the relative importance of each, and decide on a final score (O).

Produce a JSON object scores without providing any reasoning.

Example: {"M": 2, "T": 1, "O": 1}

4.4 Experimental Design

To evaluate the effectiveness of the proposed ensemble-based relevance labeling approach, we ran the system on the same dataset used in the LLM Judge Challenge. This ensured direct comparability with existing results.

The experimental workflow was structured around the four pipelines developed in this project. First, the `INGEST` pipeline was executed to normalize the dataset into our internal format. Second, the `INFER` pipeline was run sequentially with each of the five selected models to generate individual relevance judgements. Third, these judgements were combined into a final relevance label using different aggregation strategies in the `AGGREGATE` pipeline. Finally, the `EVALUATE` pipeline was used to compute the evaluation metrics reported by the LLM Judge Challenge, as well as additional agreement and classification measures.

To avoid conclusions based on single-run effects, each experimental configuration was executed three times in total, consisting of one initial run and two additional repetitions. Reported results were compared across runs to verify consistency.

While larger ensembles may increase confidence in the final vote, they also incur higher latency and cost. We therefore used a default ensemble of five models in almost all experiments. This allowed us to assess the trade-off between ensemble size and evaluation quality.

In addition to the five models used throughout most experiments, two further models (`cohere-command-r7b-12-2024` and `qwen3-8b`) were included in selected runs. These models were added to extend the ensemble to observe how additional models affect aggregation outcomes.

When aggregating model judgements, ties may occur. We considered three aggregation strategies inspired by the work of Rahmani et al. [30]: Average Voting (AV), Majority Voting with Average Tie Breaking (MVA), and Majority Voting with Random Tie Breaking (MVR). Average voting (AV) produces a single score by design, whereas majority voting approaches (MVA and MVR) may yield ties and therefore require an explicit tie-breaking rule.

Following Ma et al. [21], the temperature parameter was set to 0.01 for all models to



minimize response randomness. All models were evaluated using the prompt introduced by Thomas et al. [37], adapted to the TREC relevance scale (see listing 4.1). The prompt follows a one-shot setup, providing a query–document pair and requesting a relevance score.

4.5 Evaluation Metrics

This section introduces the evaluation metrics used to assess the agreement between automated relevance judgements and human annotations. We begin with standard classification metrics before introducing agreement-based metrics that explicitly account for chance agreement and ordinal relevance scales. The metrics differ in their formulation and underlying assumptions, but are all employed in this thesis to assess how closely automated relevance judgements align with the corresponding human labels.

Classification metrics

Common classification metrics such as accuracy, precision, and recall are typically defined for *binary* classification tasks. In binary classification, each prediction assigns an instance to be either positive or negative. The prediction can either match the ground truth or not, resulting in four possible outcomes: True Positive (TP), False Positive (FP), True Negative (TN), and False Negative (FN).

These outcomes are commonly summarized using a 2×2 confusion matrix:

Ground Truth	Prediction	
	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Using this notation, accuracy, precision, and recall are defined as follows [11]:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (4.1)$$

Accuracy measures the proportion of correctly classified instances among all instances.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4.2)$$

Precision measures the proportion of predicted positive instances that are truly positive. It reflects the reliability of positive predictions.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4.3)$$

Recall measures the proportion of truly positive instances that are correctly identified by the classifier.



Our labeling pipeline, however, constitutes a *multi-class* classification problem. Each LLM assigns one of four relevance labels (0–3) to a document, which is then compared against the corresponding human annotation. The classification outcomes can be summarized using a 4×4 confusion matrix:

Human Label	LLM Prediction			
	0	1	2	3
0	n_{00}	n_{01}	n_{02}	n_{03}
1	n_{10}	n_{11}	n_{12}	n_{13}
2	n_{20}	n_{21}	n_{22}	n_{23}
3	n_{30}	n_{31}	n_{32}	n_{33}

With the exception of accuracy, the binary classification framework is not directly applicable to the multi-class setting because the classifier no longer makes a binary decision. A common approach is to compute class-wise precision and recall. Each class is treated as the positive class in turn, while all remaining classes are grouped as negative. For each class i , precision and recall are defined as follows [35]:

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i} \quad (4.4)$$

Precision_i measures the proportion of instances predicted as class i that are correctly annotated as class i .

$$\text{Recall}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i} \quad (4.5)$$

Recall_i measures the proportion of instances annotated as class i that are correctly predicted as class i by the classifier.

In the statistical literature, class-wise precision and recall are often aggregated using micro- or macro-averaging to obtain a single F1 score [35, 11]. However, it is not required for this thesis, as class-wise metrics are used solely to analyze the labeling behavior of the ensemble for individual relevance classes.

In contrast to precision and recall, accuracy generalizes directly to the multi-class setting [11]. It is computed as the proportion of instances for which the predicted relevance label matches the human annotation, corresponding to the sum of the diagonal entries of the confusion matrix, divided by the total number of instances.

Cohen’s Kappa

Accuracy does not account for agreement that may occur by chance, which can lead to overly optimistic estimates of model performance. The statistical literature addresses this limitation by framing the task as an agreement problem between two raters. In this setting, a human rater and a LLM independently assign N items to K mutually exclusive categories.

Measures designed to quantify agreement between raters are commonly referred to as inter-rater reliability metrics, or more generally as agreement metrics. One such metric



Table 4.7: Confusion matrix for two raters with K relevance classes

		Rater B				Row sum
		Class 1	Class 2	...	Class K	
Rater A	Class 1	n_{11}	n_{12}	...	n_{1K}	$n_{1\cdot}$
	Class 2	n_{21}	n_{22}	...	n_{2K}	$n_{2\cdot}$
	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
	Class K	n_{K1}	n_{K2}	...	n_{KK}	$n_{K\cdot}$
Column sum		$n_{\cdot 1}$	$n_{\cdot 2}$	...	$n_{\cdot K}$	N

is Cohen's κ , whose primary advantage over accuracy is that it explicitly corrects for agreement expected by chance [23, 43].

Cohen's κ is defined as

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (4.6)$$

where p_o denotes the observed agreement rate between the two raters and p_e denotes the expected agreement rate by chance [23, 43].

The observed agreement p_o in eq. (4.6) is computed as

$$p_o = \frac{1}{N} \sum_{i=1}^K n_{ii}, \quad (4.7)$$

where n_{ii} denotes the number of instances assigned to relevance class i by both raters, K is the number of classes, and N is the total number of items being rated. In terms of the confusion matrix, this corresponds to the sum of the diagonal entries divided by the total number of instances. Note that this is equivalent to the accuracy measure described earlier.

The expected agreement by chance p_e in eq. (4.6) is computed as

$$p_e = \frac{1}{N^2} \sum_{i=1}^K n_{i\cdot} n_{\cdot i}, \quad (4.8)$$

where $n_{i\cdot}$ denotes the sum of row i and $n_{\cdot i}$ denotes the sum of column i of the confusion matrix.

κ takes values in the range $-1 \leq \kappa \leq 1$, where $\kappa = 1$ indicates perfect agreement, $\kappa = 0$ indicates agreement no better than chance, and negative values indicate agreement worse than chance. Landis and Koch [16] propose the qualitative interpretation shown in table 4.8.

Krippendorff's alpha

A limitation of Cohen's κ is that it treats all disagreements equally. Specifically, predicting a relevance label that differs by one level from the human judgement is penalized in the same way as a disagreement spanning multiple relevance levels. This is a consequence of κ being defined for nominal data, where class labels are unordered.

Krippendorff's α addresses this limitation by allowing the use of distance functions that account for the magnitude of disagreement between labels. In particular, α can be computed



Table 4.8: Interpretation of Cohen’s κ according to Landis and Koch [16].

κ range	Interpretation
$\kappa < 0.00$	Poor agreement
$0.00 \leq \kappa \leq 0.20$	Slight agreement
$0.21 \leq \kappa \leq 0.40$	Fair agreement
$0.41 \leq \kappa \leq 0.60$	Moderate agreement
$0.61 \leq \kappa \leq 0.80$	Substantial agreement
$0.81 \leq \kappa \leq 1.00$	Almost perfect agreement

using a distance function suitable for ordinal data, making it sensitive to how far predicted relevance labels deviate from human judgements. In addition, α generalizes to incomplete annotations and supports more than two raters. α is also a measurement of inter-rater reliability.

Krippendorff’s α is defined in terms of *observers*, *units*, and *values* [15]. Observers are the entities assigning values, units are the objects being annotated, and values are the categories drawn from a predefined measurement scale. In the context of this thesis, the observers are a human annotator and an LLM. Each unit corresponds to a document-query pair, and values represent relevance judgements on an ordinal scale ranging from 0 to 3.

In its general form, Krippendorff [15] defines α as

$$\alpha = 1 - \frac{D_o}{D_e}, \quad (4.9)$$

where D_o denotes the observed disagreement and D_e denotes the disagreement expected by chance.

The observed disagreement D_o in eq. (4.9) is computed as

$$D_o = \frac{1}{n} \sum_c \sum_k o_{ck} \text{metric} \delta_{ck}^2 \quad (4.10)$$

The expected disagreement D_e in eq. (4.9) is computed as

$$D_e = \frac{1}{n(n-1)} \sum_c n_c \sum_k n_k \text{metric} \delta_{ck}^2 \quad (4.11)$$

The computation of α is based on two intermediate representations of the annotation data: the *reliability data matrix* and the derived *coincidence matrix*. The arguments o_{ck} , n_c , n_k , and n in the two disagreement measures refer to frequencies derived from the coincidence matrix.

The function $\text{metric} \delta_{ck}$ denotes a metric-specific difference or weighting function. Its form is determined by the measurement level of the annotation scale (e.g., nominal or ordinal) and is specified at the end of the section.

The reliability data matrix is an observer-by-unit matrix that records the value assigned by each observer to each unit. Its general structure is shown in table 4.9. Missing values are permitted, allowing observers to abstain from rating individual units.



Table 4.9: General form of a reliability data matrix following Krippendorff [15] notation.

Observer	Unit 1	Unit 2	...	Unit u	...	Unit N
Observer i	c_{i1}	c_{i2}	...	c_{iu}	...	c_{iN}
Observer j	c_{j1}	c_{j2}	...	c_{ju}	...	c_{jN}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$
Observer m	c_{m1}	c_{m2}	...	c_{mu}	...	c_{mN}
Observer count	m_1	m_2	...	m_u	...	m_N

m_u is the number of observers valuing unit u . In theory, data can be missing which makes m_u variable.

From the reliability data matrix, a coincidence matrix is constructed by counting how often pairs of values occur across observers for the same unit. The general structure of such a coincidence matrix is shown in table 4.10. This matrix forms the basis for computing observed and expected disagreement, which are then used to calculate α .

Table 4.10: General form of a coincidence matrix.

Values	1	...	k	...	Row sum
1	o_{11}	...	o_{1k}	...	n_1
$\vdots$	$\vdots$	...	$\vdots$	...	$\vdots$
c	o_{c1}	...	o_{ck}	...	$n_c = \sum_k o_{ck}$
$\vdots$	$\vdots$	...	$\vdots$	...	$\vdots$
Col sum	n_1	...	n_k	...	$n = \sum_c \sum_k o_{ck}$

Indices c and k range over the set of possible values an observer may assign to a unit.

$$o_{ck} = \sum_u \frac{\text{Number of } c\text{-}k \text{ pairs in unit } u}{m_u - 1}, \quad (4.12)$$

In this thesis, the reliability data matrix consists of two observers and approximately 4,000 documents. An example instantiation for the relevance labelling task is shown in table 4.11. Each entry corresponds to a relevance score in 0, 1, 2, 3 assigned by the respective observer.

Table 4.11: Example reliability data matrix for relevance labelling in this thesis.

Observer	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5	Doc 6
Human annotator	3	1	0	2	3	2
LLM annotator	2	1	1	2	0	2
Observer count	2	2	2	2	2	2

The resulting coincidence matrix for the first six documents of the example is displayed in table 4.12.



Table 4.12: Coincidence matrix for the six-document example (unweighted pair counts).

	0	1	2	3	n_c
0	.	1	.	1	2
1	1	2	.	.	3
2	.	.	4	1	5
3	1	.	1	.	2
n_k	2	3	5	2	$n = 12$

Note that because each unit is annotated by only two observers, the computation of o_{ck} simplifies to

$$o_{ck} = \sum_u \text{Number of } c\text{-}k \text{ pairs in unit } u. \quad (4.13)$$

Krippendorff [15] gives many possible weighting functions $_{\text{metric}}\delta_{ck}^2$. The one we used for ordinal data is defined as

$$_{\text{ordinal}}\delta_{ck}^2 = \left(\sum_{g=c}^k n_g - \frac{n_c + n_k}{2} \right)^2 \quad (4.14)$$

The weighting function yields the weight matrix shown in Table 4.13, which is used to compute α for the six-document example.

Table 4.13: Weights computed with $_{\text{ordinal}}\delta_{ck}^2$ for the six-document example.

	0	1	2	3
0	0	2.5^2	6.5^2	10^2
1	2.5^2	0	4^2	7.5^2
2	6.5^2	4^2	0	3.5^2
3	10^2	7.5^2	3.5^2	0

Using the coincidence matrix from Table 4.12 and the ordinal distance weights from Table 4.13 as well as eqs. (4.9) to (4.11), the observed disagreement is

$$D_o = \frac{1 \cdot 2.5^2 + 1 \cdot 10^2 + 1 \cdot 2.5^2 + 1 \cdot 10^2 + 1 \cdot 3.5^2 + 1 \cdot 3.5^2}{12} = 19.75.$$

The expected disagreement evaluates to

$$\begin{aligned} D_e = \frac{1}{12 \cdot 11} & \left(2(3 \cdot 2.5^2 + 5 \cdot 6.5^2 + 2 \cdot 10^2) \right. \\ & + 3(2 \cdot 2.5^2 + 5 \cdot 4^2 + 2 \cdot 7.5^2) \\ & + 5(2 \cdot 6.5^2 + 3 \cdot 4^2 + 2 \cdot 3.5^2) \\ & \left. + 2(2 \cdot 10^2 + 3 \cdot 7.5^2 + 5 \cdot 3.5^2) \right) \approx 23.64. \end{aligned}$$



Substituting into the definition of α yields

$$\alpha = 1 - \frac{D_o}{D_e} \approx 0.17.$$

α takes values in the range $-1 \leq \alpha \leq 1$, where $\alpha = 1$ indicates perfect agreement, $\alpha = 0$ indicates agreement no better than chance, and negative values indicate systematic disagreement. Krippendorff proposes the interpretation shown in table 4.14 [22].

Table 4.14: Interpretation of Krippendorff's α .

α value	Interpretation
$\alpha = 1.00$	Perfect agreement
$\alpha \geq 0.80$	Reliable agreement
$0.67 \leq \alpha < 0.80$	Tentative conclusions possible
$0.00 \leq \alpha < 0.67$	Agreement insufficient for reliable conclusions
$\alpha = 0.00$	No agreement beyond chance
$\alpha < 0.00$	Systematic disagreement



Chapter 5

Experimental Results

This chapter reports the empirical results of the proposed ensemble-based relevance labelling approach. It first summarizes overall runtime characteristics and assesses run-to-run stability across repeated configurations. It then benchmarks the ensemble against the LLM Judge Challenge leaderboard and a strong single-model reference, highlighting cost-latency trade-offs and the effect of ensemble size and aggregation strategy. Finally, it analyzes misclassification patterns to identify differences in labelling behavior across relevance classes.

5.1 Run Statistics and Reproducibility

In total, the system produced 74,000 LLM relevance judgements. Out of these, 49 outputs (0.07%) were unparseable and could not be assigned a label. The total inference time across all runs was approximately 62 hours. The total monetary cost was 27 USD, corresponding to 0.000369 USD per judgement, well below one cent per label.

To reduce the influence of run-to-run variability, each experimental configuration was executed three times whenever possible. Repeated runs allow us to assess the stability and reproducibility of model behaviour and ensure that reported results are not driven by single-run artefacts.

Table 5.1 reports intra-model agreement across repeated runs, measured using Krippendorff’s α . High agreement values for most models indicate that individual models produce highly consistent relevance judgements across runs.

Figure 5.1 further illustrates this stability using Cohen’s κ , computed for each run by comparing LLM-assigned relevance labels against human annotations. For nearly all models, fluctuations across runs are negligible, with deviations of at most 0.009. The only notable exception is llama-3.2-3b, which exhibits substantially lower self-agreement and is treated as an outlier in this regard.

For `phi-4-multimodal-instruct`, only two runs are reported, as this model was removed

Table 5.1: Intra-model agreement across repeated runs measured with Krippendorff’s α .

Model	α
gemma-3-4b	0.991
ministral-3b	0.940
phi-4-multimodal-instruct	0.992
llama-3.2-3b-instruct	0.533
ui-tars-1.5-7b	0.967
GPT-5.1	0.915



from the OpenRouter API before a third repetition could be completed.

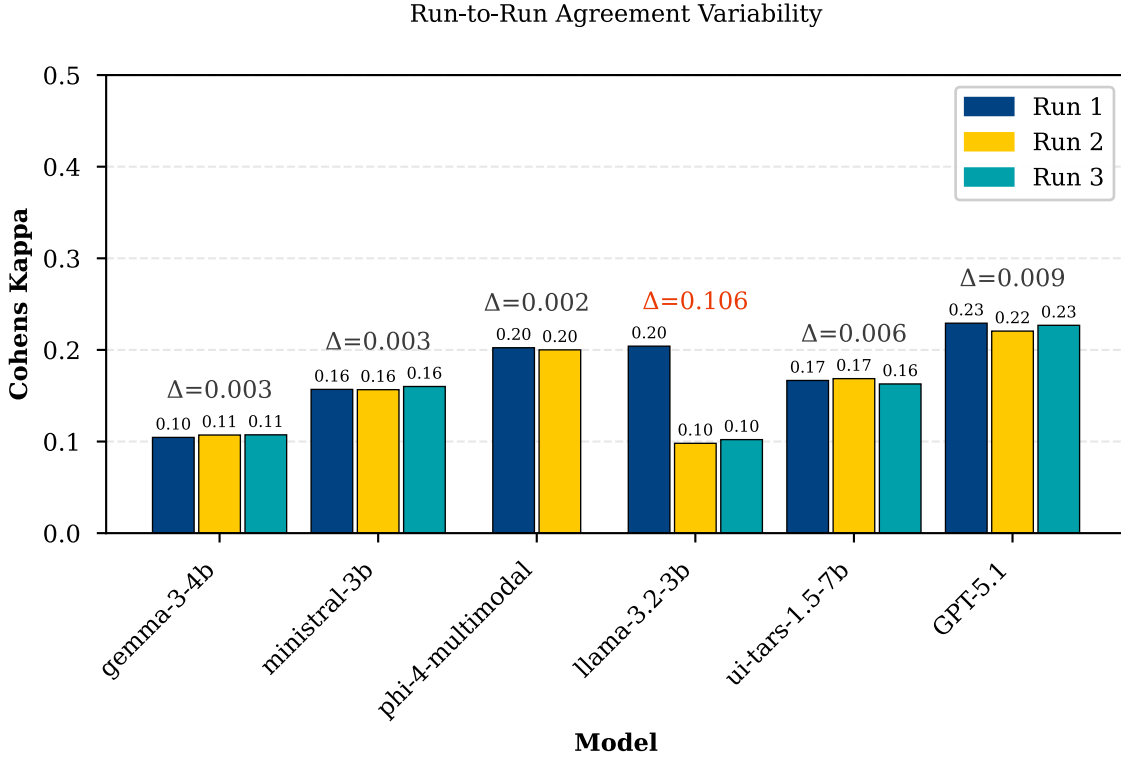


Figure 5.1: Cohen’s κ for each model across repeated runs, computed against human annotations.

5.2 LLM Judge Challenge Ranking

Table 5.2 compares our ensemble-based approach to 42 runs from the LLM Judge Challenge. Using MVA and MVR as aggregation strategies, the ensemble ranks 19th and 20th according to Cohen’s κ , and 18th and 21st according to Krippendorff’s α . Out of 46 total runs, this places both approaches close to the median of the ranking.

In contrast, the ensemble using AV performs substantially worse, ranking 35th and 29th under the same metrics, which places it in the lower third of submissions. The reference run using GPT 5.1 as a state-of-the-art single-model baseline achieves stronger results, ranking 13/46 according to Cohen’s κ and 8/46 according to Krippendorff’s α , corresponding to an upper-quartile placement under Krippendorff’s α .

Figure 5.2 compares agreement with human annotations measured using Cohen’s κ for individual models, ensemble variants, and the GPT 5.1 reference model, all evaluated on the LLM Judge Challenge dataset. For individual models ordered by parameter count, Cohen’s κ values vary across model sizes. No clear monotonic trend is visible as the number of parameters increases.



Table 5.2: LLM Judge Challenge rankings with our approaches highlighted.

Ranked by Cohen’s κ			Ranked by Krippendorff’s α		
Rank	Submission	κ	Rank	Submission	α
1	willia-umbrela1	0.2863	1	Olz-gpt4o	0.5020
2	h2oloo-zeroshot1	0.2817	2	h2oloo-fewself	0.4958
3	h2oloo-fewself	0.2774	3	willia-umbrela1	0.4918
4	willia-umbrela3	0.2741	4	llmjudge-thomas3	0.4877
5	willia-umbrela2	0.2688	5	RMITIR-llama70B	0.4873
6	RMITIR-llama70B	0.2654	6	llmjudge-cot3	0.4870
7	Olz-gpt4o	0.2625	7	h2oloo-zeroshot1	0.4812
8	h2oloo-zeroshot2	0.2589	8	GPT 5.1	0.4752
9	Olz-exp	0.2519	9	Olz-exp	0.4701
10	Olz-multiprompt	0.2445	10	llmjudge-simple3	0.4642
11	RMITIR-GPT4o	0.2388	11	willia-umbrela2	0.4556
12	llmjudge-thomas3	0.2293	12	Olz-multiprompt	0.4551
13	GPT 5.1	0.2292	13	Olz-halfbin	0.4536
14	llmjudge-cot3	0.2271	14	willia-umbrela3	0.4535
15	llmjudge-simple3	0.2110	15	Olz-somebin	0.4471
16	Olz-somebin	0.2109	16	RMITIR-GPT4o	0.4108
17	TREMA-sumdecompose	0.2088	17	prophet-setting1	0.4069
18	Olz-halfbin	0.2064	18	Ensemble (MVA)	0.3942
19	Ensemble (MVA)	0.2061	19	TREMA-sumdecompose	0.3926
20	Ensemble (MVR)	0.2051	20	h2oloo-zeroshot2	0.3898
21	RMITIR-llama38b	0.2006	21	Ensemble (MVR)	0.3882
22	TREMA-CoT	0.1961	22	NISTRetrieval-reason0	0.3874
23	NISTRetrieval-instruct2	0.1880	23	NISTRetrieval-reason2	0.3874
24	NISTRetrieval-instruct0	0.1877	24	RMITIR-llama38b	0.3873
25	NISTRetrieval-instruct1	0.1874	25	NISTRetrieval-reason1	0.3872
26	NISTRetrieval-reason1	0.1845	26	TREMA-all	0.3855
27	NISTRetrieval-reason0	0.1844	27	llmjudge-thomas2	0.3853
28	NISTRetrieval-reason2	0.1838	28	TREMA-CoT	0.3852
29	TREMA-4prompts	0.1829	29	Ensemble (AV)	0.3822
30	prophet-setting1	0.1823	30	NISTRetrieval-instruct2	0.3821
31	prophet-setting2	0.1757	31	NISTRetrieval-instruct0	0.3819
32	TREMA-direct	0.1742	32	NISTRetrieval-instruct1	0.3812
33	TREMA-naiveBdecompose	0.1741	33	TREMA-direct	0.3729
34	llmjudge-thomas2	0.1723	34	llmjudge-simple2	0.3672
35	Ensemble (AV)	0.1645	35	TREMA-naiveBdecompose	0.3579
36	llmjudge-cot2	0.1560	36	llmjudge-cot2	0.3263
37	TREMA-all	0.1471	37	llmjudge-cot1	0.3218
38	prophet-setting4	0.1471	38	llmjudge-thomas1	0.3207
39	TREMA-other	0.1408	39	TREMA-questions	0.3148
40	llmjudge-simple2	0.1327	40	prophet-setting2	0.3144
41	llmjudge-cot1	0.1284	41	TREMA-4prompts	0.2888
42	llmjudge-thomas1	0.1236	42	llmjudge-simple1	0.2808
43	TREMA-questions	0.1137	43	TREMA-other	0.2712
44	TREMA-rubric0	0.0779	44	TREMA-nuggets	0.1691
45	llmjudge-simple1	0.0754	45	prophet-setting4	0.1623
46	TREMA-nuggets	0.0604	46	TREMA-rubric0	0.1036

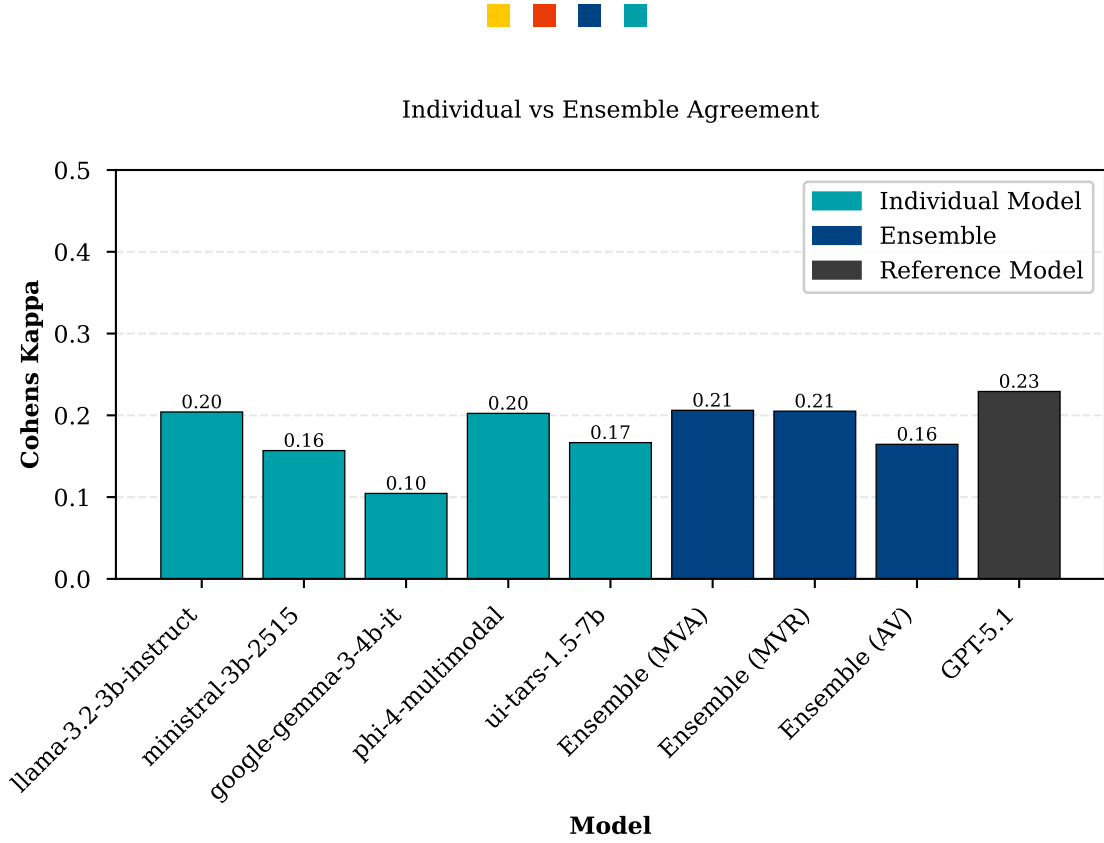


Figure 5.2: Agreement between individual models, the ensemble, and the reference model with human annotations, measured using Cohen’s κ . Individual models are sorted from left to right by parameter count.

5.3 Cost-Latency Trade-offs

Figure 5.3 compares the total inference cost and total latency of the reference model (GPT 5.1) with the ensemble-based approach.

The reference model incurs a total cost of 6.84 USD and a total latency of approximately 4.6 hours. In comparison, the ensemble incurs a combined cost of 1.32 USD and a total latency of approximately 4.8 hours. The ensemble latency reflects the cumulative contribution of its constituent models, each processing the full set of judgements independently.

Figure 5.3 further decomposes the ensemble’s cost and latency into contributions from its individual models. Across ensemble components, total costs range from 0.11 USD to 0.51 USD, while total latencies range from approximately 0.75 to 1.31 hours per model. No single ensemble component dominates either total cost or total latency, and both quantities are distributed across the constituent models.

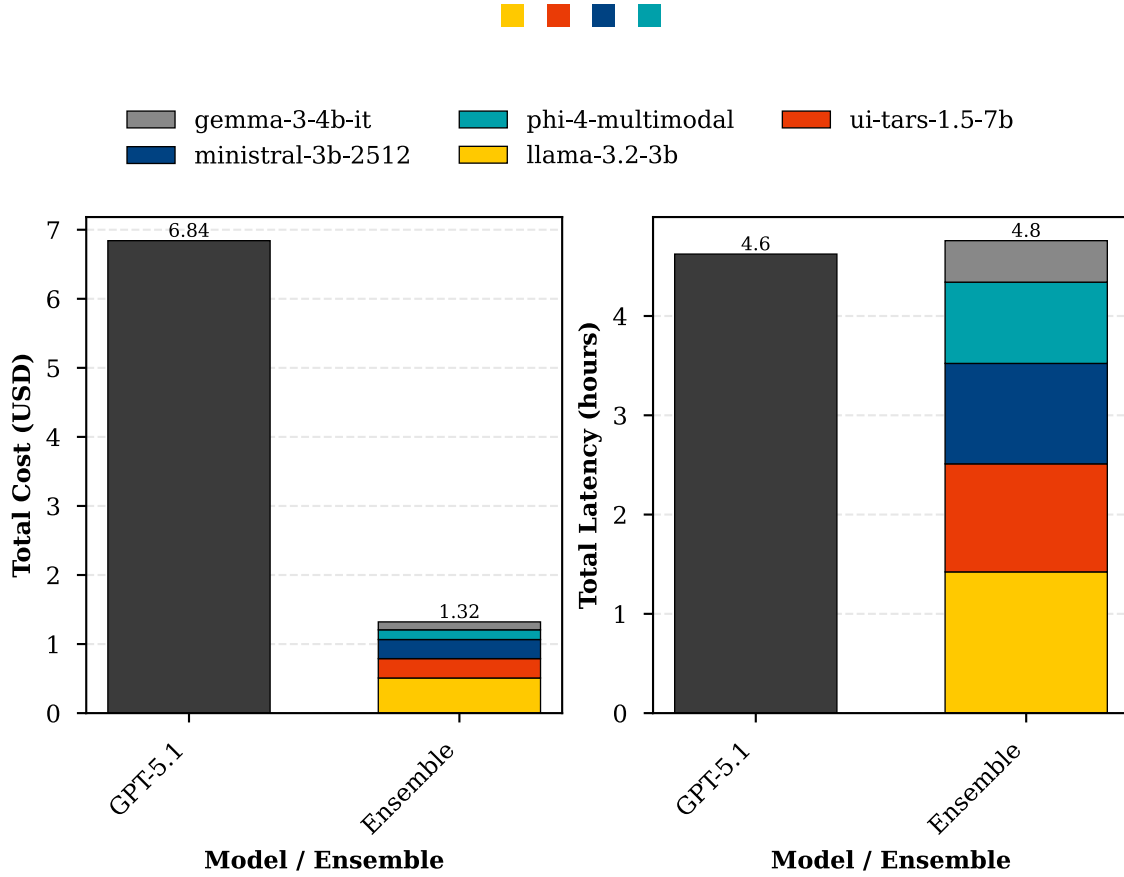


Figure 5.3: Total cost and latency for the reference model and the ensemble.

5.4 Ensemble Size and Aggregation Strategies

Figure 5.4 and Figure 5.5 report inter-rater reliability measured using Cohen’s κ for ensembles of increasing size. Ensemble size denotes the number of models whose relevance judgements are aggregated. Three aggregation strategies are shown: AV, MVA, and MVR.

The two figures differ in how ensemble members are added. In the *small-to-big* configuration, aggregation starts with the smallest model and progressively adds larger models by parameter count. In the *big-to-small* configuration, aggregation starts with the largest model and progressively adds smaller models.

For both aggregation orders, majority-vote based strategies (MVA and MVR) consistently yield higher Cohen’s κ values than average voting for most ensemble sizes. The two majority voting variants follow similar trajectories, with small differences in absolute Cohen’s κ values across ensemble sizes.

In fig. 5.4, both majority-voting strategies (MVA and MVR) increase in Cohen’s κ up to an ensemble size of four, after which no further systematic increase is observed. In fig. 5.5, a similar pattern is visible, with Cohen’s κ values for both majority-voting strategies increasing up to an ensemble size of three and remaining within a comparable range for larger ensemble sizes.

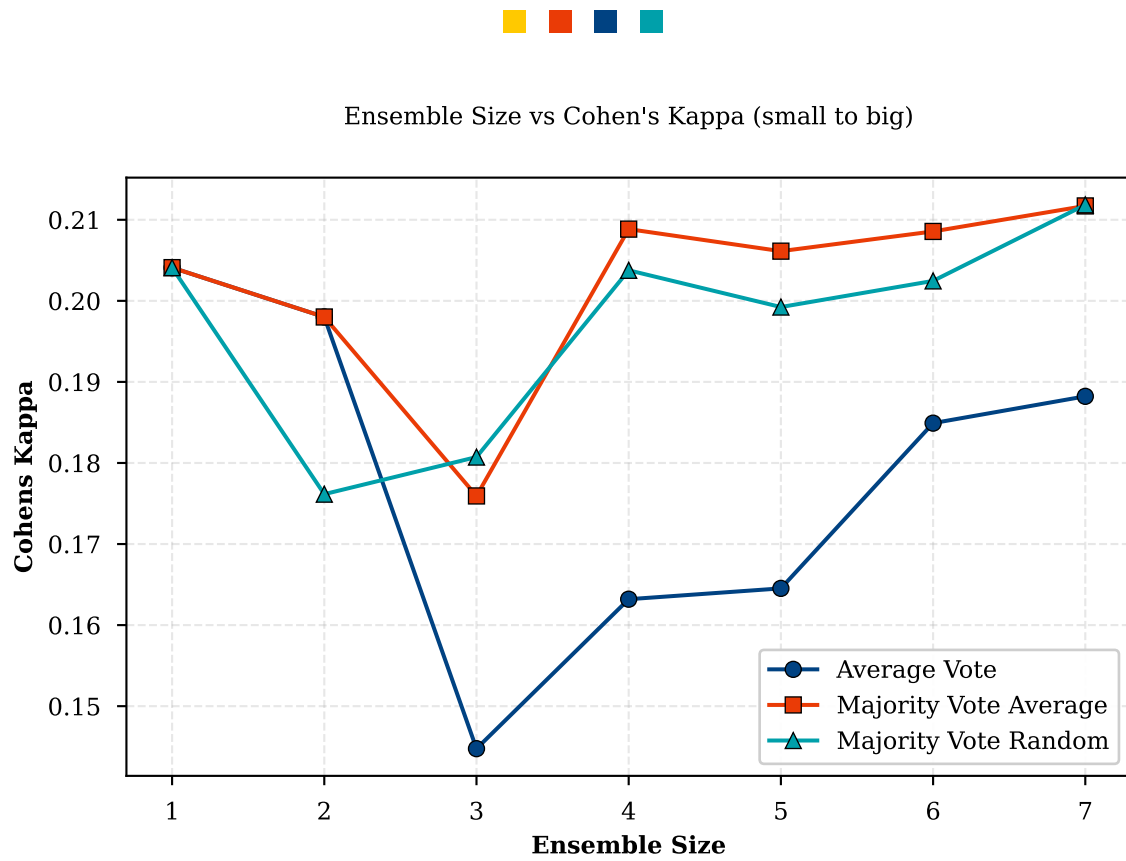


Figure 5.4: Cohen's κ as a function of ensemble size when incrementally adding bigger models, starting from the smallest model.

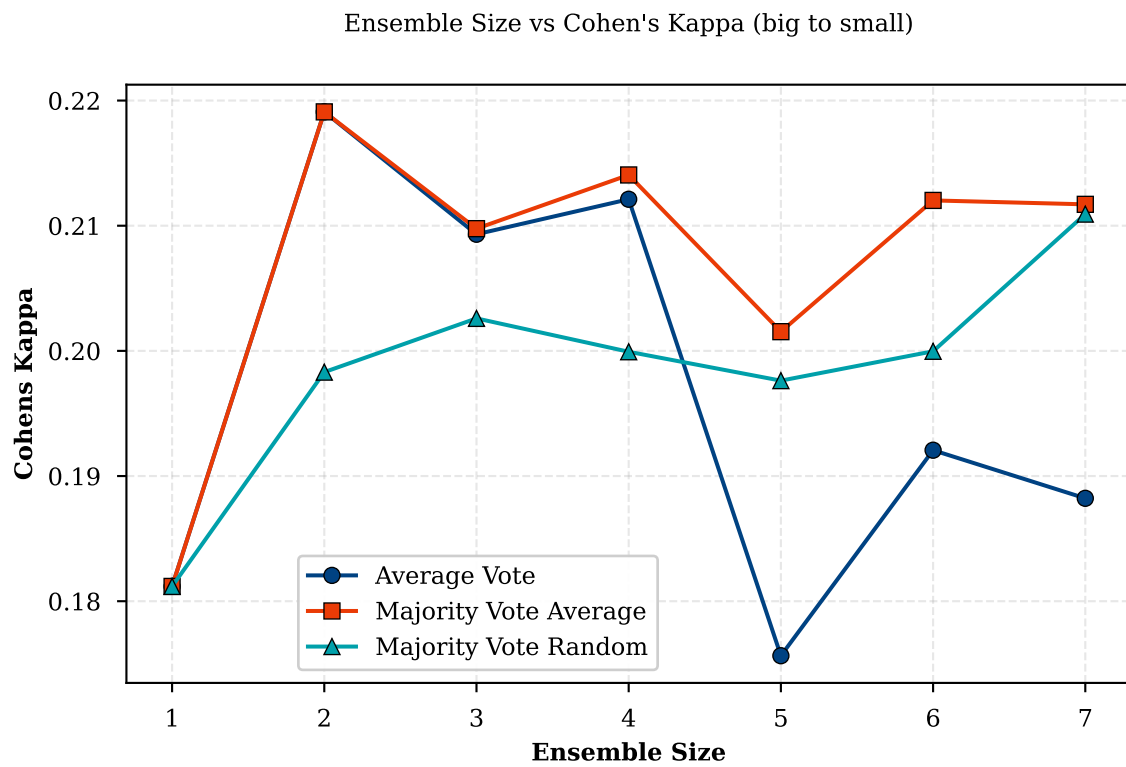


Figure 5.5: Cohen's κ as a function of ensemble size when incrementally adding smaller models, starting from the largest model.



5.5 Misclassification Analysis

Following Thomas et al. [37], we distinguish between soft disagreement (a disagreement between two relevance judgements differing by one adjacent relevance level, appearing in cells adjacent to the main diagonal of the confusion matrix) and hard disagreement (a disagreement between relevance extremes, appearing in the top-right and bottom-left corner cells of the confusion matrix).

Figure 5.6 and Figure 5.7 show the confusion matrices / heatmaps for the ensemble-based MVA approach and the GPT 5.1 reference run, respectively. In both cases, most predictions lie along the main diagonal, indicating that both approaches distinguish broadly between relevant and irrelevant documents. Soft disagreements are common in both matrices.

For the ensemble-based approach, hard disagreements are infrequent but asymmetric. Predictions of high relevance for documents annotated as irrelevant (top right corner) occur more often than predictions of irrelevance for perfectly relevant documents (bottom left corner). In the GPT 5.1 reference run, hard disagreements are less frequent overall.

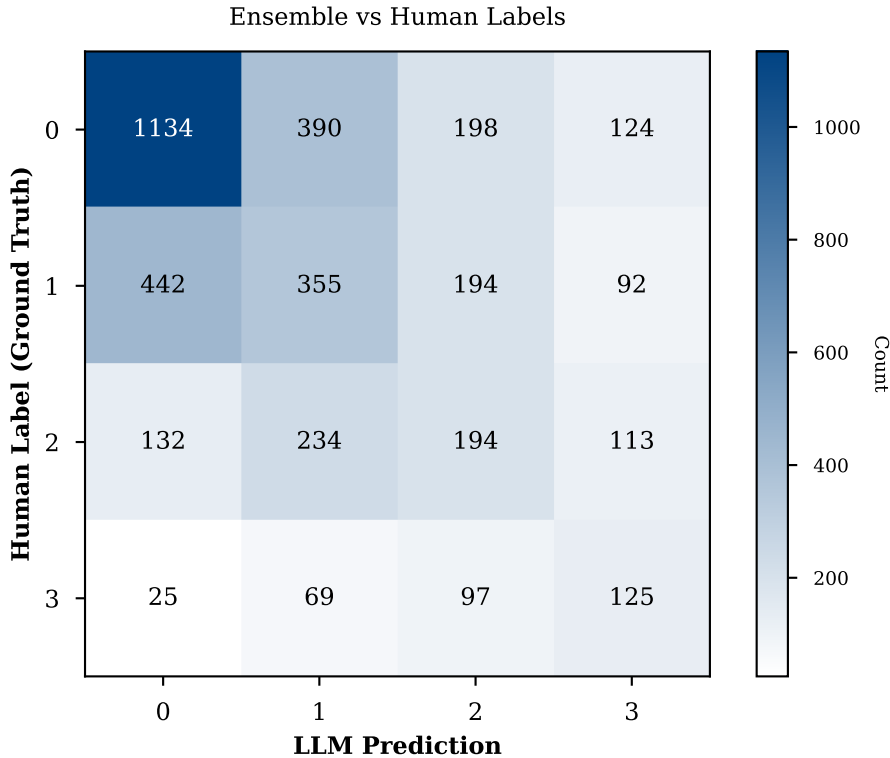


Figure 5.6: Heatmap of ensemble-based relevance labels (MVA) compared to human labels.

Table 5.3 reports class-wise classification metrics derived from the confusion matrices. Overall accuracy is similar for both approaches, differing by less than two percentage points. Across relevance classes, recall and precision vary substantially. The ensemble achieves higher recall for the irrelevant class, while the GPT 5.1 reference run shows higher recall for the relevant, highly relevant, and perfectly relevant classes. Precision exhibits a similar pattern, with GPT 5.1 attaining higher precision for higher relevance classes.

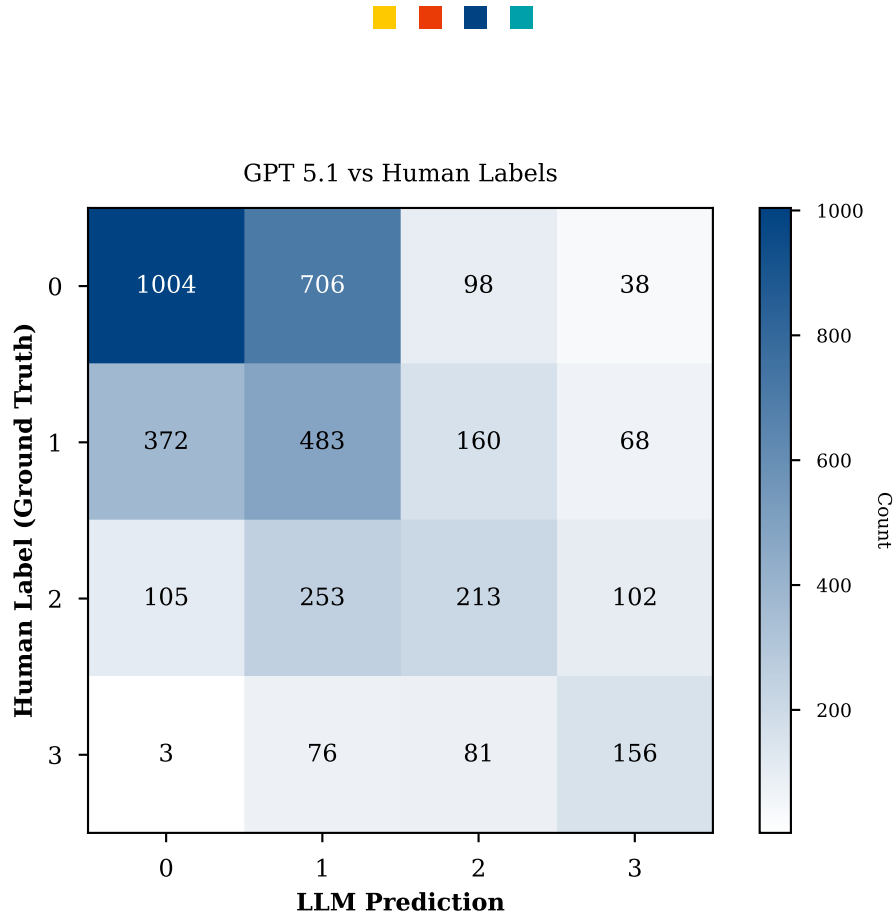


Figure 5.7: Heatmap of GPT 5.1 relevance labels compared to human labels.

Table 5.3: Confusion Matrix Comparison: Ensemble (MVA) vs GPT-5.1

Metric	Ensemble	GPT-5.1
Accuracy	46.1%	47.4%
Recall (Irrelevant)	61.4%	54.4%
Recall (Relevant)	32.8%	44.6%
Recall (Highly Relevant)	28.8%	31.6%
Recall (Perfectly Relevant)	39.6%	49.4%
Precision (Irrelevant)	65.4%	67.7%
Precision (Relevant)	33.9%	31.8%
Precision (Highly Relevant)	28.4%	38.6%
Precision (Perfectly Relevant)	27.5%	42.9%
Over-predicting (LLM > Human)	28.4%	29.9%
Under-predicting (LLM < Human)	25.5%	22.7%
Soft Disagreement	37.5%	42.7%
Hard Disagreement	16.3%	9.9%



Chapter 6

Discussion

This chapter interprets the experimental findings in light of the research questions and situates them within prior work on LLM-based relevance judging. It discusses what the results imply about ensemble label quality, aggregation behaviour, and cost-latency trade-offs, and highlights where the evidence contradicts the initial hypotheses. The chapter then outlines key limitations that affect how broadly the findings can be generalized. Finally, it derives practical implications for deployment at donista and identifies directions for future work.

6.1 Interpretation of Results

Labelling Quality of the Ensemble – RQ1

To assess the overall labelling quality of the proposed ensemble-based approach, we compare its performance to the submissions of the LLM Judge Challenge reported in Table 5.2. Since both MVA and MVR rank close to the median of all submissions, we conclude that the ensemble achieves labelling quality close to the median of other LLM-based judging systems. Related work by Rahmani et al. [30] reports stronger results for a conceptually similar approach. Their system combines multiple prompts and model judgements and ranks within the top third of all LLM Judge Challenge submissions.

While this comparison suggests competitive performance relative to other LLM judges, it does not address how closely the ensemble aligns with human judgements. This question is central to evaluating the practical validity of the LLM-as-a-Judge paradigm. As shown in Figure 5.2, the highest Cohen’s κ values achieved by the ensemble are approximately 0.206. According to the interpretation guidelines in Table 4.8, this corresponds to agreement at the boundary between slight and fair agreement. In addition, table 5.3 reports an overall accuracy of 46%, indicating that fewer than half of the predicted relevance labels match the human annotations exactly. This level of agreement remains below what would be desirable for reliable automated relevance labelling.

For context, Thomas et al. [37] report Cohen’s κ values of approximately 0.2 for their weakest runs and up to 0.64 for their strongest runs. They further report κ values of around 0.5 for agreement between human raters. The authors of the *William-umbrela1* submission to the LLM Judge Challenge note that other systems in the literature achieve κ values of up to 0.4 and, in some cases, up to 0.6, which is consistent with these findings. They



conclude that the LLM Judge Challenge dataset is particularly challenging or noisy.

Given that none of the LLM Judge Challenge submissions achieve κ values exceeding 0.28, this interpretation appears plausible. Consequently, caution is warranted when drawing conclusions about the overall viability of the LLM-as-a-Judge framework based solely on performance within the LLM Judge Challenge.

Further insight is provided by the heatmap in fig. 5.6. A large fraction of misclassifications occurs near the main diagonal, indicating that many predictions differ from the human labels by only one relevance level. These soft disagreements contribute to the low agreement scores, but do not fully account for them.

The distribution of labels in the heatmap also reveals a strong concentration of judgements in the lower relevance categories. Both human annotations and LLM predictions are most frequent for relevance levels 0 and 1, which is visible in the higher density of counts in the corresponding regions of the matrix. This alignment suggests that, at a coarse level, the ensemble captures the overall relevance distribution similarly to human assessors, even though finer-grained distinctions between adjacent relevance levels remain challenging.

Overall, we conclude that the ensemble exhibits coarse alignment with human relevance judgements, but struggles with consistent fine-grained differentiation between adjacent relevance levels. At the same time, the dataset appears to contain a non-trivial amount of annotation noise, which likely contributes to the low agreement scores. This introduces a fundamental challenge for evaluation: systems that diverge from human judgements by correcting inconsistent or erroneous labels may be penalized, while systems that reproduce the same annotation errors may be rewarded. This limitation should be kept in mind when interpreting the following research questions.

Impact of Ensemble Aggregation on Hard Disagreements – SRQ1

The initial hypothesis in SRQ1 assumed that hard disagreements primarily result from model-specific misunderstandings of the query. Under this assumption, different LLMs were expected to fail in different ways. Aggregating their relevance judgements should therefore cancel out such errors and reduce the number of hard disagreements.

The results contradict this expectation. As shown in Table 5.3, the ensemble exhibits a substantially higher rate of hard disagreements than the GPT 5.1 reference run (16.3% vs. 9.9%). This increase occurs despite comparable overall accuracy and a lower proportion of soft disagreements.

Further insight is provided by the heatmaps in Figure 5.6 and Figure 5.7. Compared to the reference model, the ensemble-based approach more frequently assigns high relevance scores to documents annotated as irrelevant. Hard disagreements caused by overestimation of relevance occur substantially more often than hard disagreements caused by underestimation. This asymmetric error pattern suggests a systematic tendency of the ensemble to overestimate relevance.

Overall, the ensemble does not reduce hard disagreements in this setting. On the contrary, it increases them. This finding challenges the assumption that model diversity is sufficient to mitigate severe misclassifications in LLM-based relevance labeling.

One possible explanation is that the ensemble members exhibit correlated errors. Although the models were selected to be diverse in terms of architecture and intended use cases, they



are all relatively small and may share similar limitations. If multiple models fail in similar ways, aggregation is unlikely to provide corrective benefits and may instead reinforce these errors. This aligns with the increase in hard disagreements that we found.

A second possible explanation is that the state-of-the-art reference model adheres more reliably to the provided instructions. In particular, the model was explicitly instructed to weigh relevance signals such as query intent and document trustworthiness and to derive a final relevance score based on this assessment. The reference model may be better calibrated or better able to follow these instructions consistently, resulting in judgements that align more closely with human annotations. However, no direct evidence is available to substantiate this interpretation, and it therefore remains speculative.

Cost and Latency of the Ensemble – RQ2

As shown in Figure 5.3, the ensemble-based approach achieves a substantial reduction in inference cost compared to the GPT 5.1 reference run, while exhibiting a comparable end-to-end latency under sequential execution. The observed cost difference corresponds to a reduction of approximately 81%, showing that aggregating multiple smaller models can be significantly more cost-efficient than relying on a single large model.

The similar latency profiles of about 5 hours observed in Figure 5.3 are primarily a consequence of the sequential execution strategy used for the ensemble. Since each ensemble component processes the full set of documents independently, the total runtime reflects the sum of per-model latencies. Under a fully parallel execution model, the effective ensemble latency would instead be bounded by the slowest individual model. Based on the measured per-model runtimes, this would reduce the ensemble latency to approximately 1.42 hours, corresponding to a speedup of roughly $3.3\times$ relative to the GPT 5.1 reference run.

Although the monetary cost differences are pronounced, the absolute costs remain modest at the evaluated scale. In practical deployments, inference cost therefore represents only one aspect of the overall trade-off. Development effort and operational complexity must also be considered, as designing, maintaining, and validating an ensemble-based labeling pipeline introduces additional engineering overhead compared to a single-model solution.

Impact of Ensemble Size on Label Quality – SRQ2

At first glance, the small-to-big aggregation (fig. 5.4) suggests an upward trend in inter-rater reliability once ensemble size exceeds three members. Agreement is relatively unstable for ensembles of one or two models, whereas larger ensembles exhibit a more consistent increase in agreement. This observation appears consistent with the expectation underlying SRQ2, namely that aggregating a larger number of models promotes more stable, consensus-based relevance judgements.

However, this interpretation is challenged by the corresponding big-to-small aggregation (fig. 5.5). When aggregation begins with the largest models, increases in agreement from adding further ensemble members are less pronounced if present at all. Beyond an ensemble size of four, κ values remain largely stable, exhibiting only minor fluctuations across both majority voting strategies.

This contrast indicates that the upward trend observed in the small-to-big configuration is primarily driven by the order in which models are introduced. Early gains in agreement



coincide with the inclusion of larger models, rather than with ensemble growth itself.

Across all ensemble sizes and aggregation strategies, κ values remain within a relatively narrow range, fluctuating approximately between 0.18 and 0.22. The magnitude of these fluctuations is small, indicating that changes in ensemble size are associated with only modest variations in inter-rater reliability. Taken together, these results suggest that ensemble size alone is not the dominant factor shaping inter-rater reliability.

Impact of Aggregation Strategy on Label Quality – SRQ3

Across all evaluated settings, AV yields lower agreement with human annotations than both majority voting strategies. As shown in Table 5.2, AV ranks below MVA and MVR. Similarly, Figure 5.2 shows that AV achieves a maximum Cohen’s κ of approximately 0.165, compared to approximately 0.206 for the majority voting strategies. Notably, that also means the ensemble with MVA exhibits lower Cohen’s κ values than two of the individual models. This pattern is consistent across incremental ensemble configurations in both the small-to-big and big-to-small settings (Figures 5.4 and 5.5), where AV consistently produces lower agreement scores than majority voting. By contrast, the two majority voting strategies behave very similarly. Across all reported tables and figures, differences in agreement between MVA and MVR are negligible.

One plausible contributing factor is the distribution of relevance labels in the dataset. As shown in Table 4.1, the dataset is skewed toward irrelevant documents, with label 0 accounting for approximately 45% of all qrels. The heatmaps in Figure 5.6 and table 5.3 further show that the ensemble tends to overestimate relevance. In such a setting, average voting - by pulling scores toward the center of the label scale - may exacerbate misalignment with human judgements by dampening strong signals that majority voting preserves.

All in all, these results suggest that average voting is less effective than majority voting for aggregating relevance judgements in this setting. However, this conclusion is based on experiments conducted on a single dataset. In addition, this finding clashes directly with results reported by Rahmani et al. [30]. They observe that AV outperforms both MVA and MVR in a conceptually similar framework on the same dataset when evaluated using Krippendorff’s α . As a result, the observed disadvantage of AV should be interpreted with caution.

Overall, these results do not permit a definitive conclusion that average voting is generally unsuitable for aggregating relevance judgements. However, the consistent and non-negligible differences observed between aggregation strategies demonstrate that aggregation choice materially affects label quality. Aggregation strategy should therefore be treated as a deliberate design decision and warrants careful consideration in ensemble-based relevance labelling systems.

6.2 Limitations

A primary limitation of this work is that the ensemble-based labeling approach is evaluated on a single dataset. This restricts the generalizability of the findings and limits the strength of conclusions regarding the relative effectiveness of different aggregation strategies. In particular, the observed differences between aggregation methods may be dataset-specific. It is plausible that no aggregation strategy is universally optimal and that effectiveness



depends on properties such as label distribution, task difficulty, or annotation noise.

Although care was taken to select a diverse set of models, the ensemble is constrained to relatively small models in terms of parameter count and those that are available through the OpenRouter platform. This design choice may introduce biases, as larger or proprietary models and models hosted outside of OpenRouter are not represented. Consequently, the results may not fully reflect the behaviour of more heterogeneous or higher-capacity ensembles.

The study further inherits known limitations of the LLM-as-a-Judge paradigm, particularly sensitivity to prompt design. Prompt variation was deliberately not explored in this work. All experiments rely on a single prompt with fixed instructions and examples. As a result, the reported outcomes may be influenced by prompt-specific biases, and alternative prompt formulations could lead to different agreement patterns.

An additional limitation arises from the execution environment. OpenRouter explicitly states that requests may be dynamically routed to maintain availability. As a consequence, individual inference requests may be served by different model instances despite specifying the same model identifier. This effect is likely responsible for the anomalous behaviour observed in the first `llama-3.2-3b` run shown in Figure 5.2, where agreement scores differ substantially from subsequent runs. This limitation is unlikely to affect the central objective of the thesis, which is not to benchmark individual models but to compare ensemble-based aggregation against a state-of-the-art reference model.

Further limitation concerns the temporal scope of the evaluated models. All models used in this thesis represent a snapshot of LLM capabilities from late 2024 to 2025. As model capabilities continue to improve, the relative effectiveness of ensemble-based aggregation and the observed agreement patterns may change. Consequently, the findings should be interpreted as reflective of current-generation models rather than as definitive conclusions about future LLM systems.

Finally, most analyses in this thesis treat human relevance judgements as the ground truth. This choice implicitly defines agreement with human annotations as the upper bound of achievable performance. An ensemble that disagrees with human labels - even when identifying annotation errors - is penalized and not rewarded. As discussed earlier, several indicators suggest that the human relevance judgements in the dataset are themselves imperfect, which limits the interpretability of agreement-based results.

6.3 Practical Implications

Building on these findings, the following section translates the empirical results into practical considerations for the design and deployment of automated relevance labelling systems.

General Practical Implications

The experimental results indicate that an ensemble of small models does not outperform a strong single-model baseline in terms of agreement with human relevance annotations. Across all evaluated settings, the state-of-the-art reference model exhibits higher alignment with human judgements. In addition, it shows fewer severe misclassifications, as reflected by the lower proportion of hard disagreements. If cost and latency are not primary constraints,



these findings suggest that employing a single, highly capable model is preferable when the primary objective is label quality.

With respect to ensemble configuration, we find no clear evidence that increasing ensemble size beyond a small number of models yields substantial gains. Contrary to the initial expectation, agreement values stabilize once the ensemble reaches four members, with only minor fluctuations observed for larger ensembles. This suggests diminishing returns from adding additional models.

Finally, while differences between aggregation strategies are modest in absolute terms, they are consistent across all observed measurements. This confirms that aggregation strategy does play a role in determining label quality and should not be treated as an incidental implementation detail. Based on the observed results, MVA emerges as a reasonable default choice.

Practical Implications for donista

A central objective of this thesis was to derive actionable recommendations for the deployment of automated relevance labelling within donista.

A key design decision for the relevance labelling system is whether to rely on a single, highly capable LLM or to employ the ensemble-based approach proposed in this thesis. The following discussion therefore first weighs the trade-offs between both approaches. In case an ensemble-based approach is selected, the section then provides concrete design recommendations derived from the experimental findings and practical experience gained over the course of this thesis.

Cost-Quality Trade-off. The experiments show that the ensemble achieves slightly lower agreement with human annotations than the state-of-the-art reference model. This results in a clear trade-off. A single large model produces higher-quality labels at higher cost, while the ensemble yields slightly weaker but still competitive labels at a fraction of the price.

Self-hosting. If self-hosting is desired, smaller models are considerably easier to deploy, which favours an ensemble approach. Hosting a state-of-the-art model typically requires substantial hardware resources. In practice, however, self-hosting is generally not advisable, as the inference costs observed through OpenRouter were low at the evaluated scale.

Developer Effort. An ensemble-based solution increases implementation complexity. Multiple models must be queried, monitored, and aggregated. In practice, this results in roughly twice the amount of code compared to a single-model pipeline.

Robustness. Ensembles improve robustness by reducing dependence on a single model endpoint. If individual models become unavailable or are removed, the system can continue operating with a reduced ensemble size. Degraded capacity can be detected and reported without interrupting the labelling pipeline.

Runtime. Under sequential execution, both approaches exhibit comparable end-to-end latency. At the evaluated scale, runtime therefore does not favour either design.

Given that most of the ensemble infrastructure is already implemented and not overly complex, the approach is justifiable from a development perspective. Importantly, the ensemble pipeline is a superset of the single-model solution. It can be configured to bypass



aggregation and rely on a single capable model if necessary. Overall, the results indicate that an ensemble-based approach is a viable and economically attractive option for relevance labelling in the context of donista.

Drawing on both the experimental findings and practical experience with the LLM-as-a-Judge framework, particularly when using OpenRouter, the following design recommendations are proposed in case the ensemble-based approach is selected.

Execution Strategy. Optimizing for parallel execution appears premature at the evaluated scale. Sequential execution avoids rate-limiting issues and remains practical given the modest absolute runtime. In a production setting, overnight batch processing is the most realistic scenario.

Aggregation Strategy. The experiments indicate that aggregation strategy has a measurable impact on label quality and should therefore be considered carefully. In the evaluated setting, majority voting has emerged as a robust default. More elaborate schemes, such as weighted voting based on estimated model capability, may offer additional improvements, but require further investigation.

Dynamic Ensemble Composition. Since OpenRouter exposes model metadata programmatically, ensemble composition can be adjusted dynamically. The provided endpoints expose a range of informative model attributes, enabling automated and informed selection of suitable models. This reduces manual maintenance effort when models are deprecated, replaced, or change in availability.

Ensemble Size. An ensemble size of four models appears sufficient in terms of label quality, as adding further models did not yield significant improvements in the experiments. However, given that inference costs remain in the range of only a few cents, adding several additional models is still advisable to increase overall system robustness. Selecting ensemble members programmatically further improves robustness against model failures and endpoint changes.

Use of Free Models. An ensemble-based pipeline naturally aligns with the use of free models offered by OpenRouter. If it is assumed that OpenRouter prices its most popular and capable models, the set of free models is likely to consist of smaller or less capable alternatives. A hybrid strategy is therefore conceivable, in which a portion of samples is labelled using an ensemble of free models, while the remaining samples are processed by a state-of-the-art model. This approach offers a balanced compromise across cost, label quality, and robustness. Of course, the main drawback of this approach is again increased development effort.

Prompt Design. Prompt sensitivity remains a known challenge in LLM-based evaluation. Given the unpredictability of prompt effects reported in the literature, retaining a well-tested prompt - such as the formulation proposed by Thomas et al. [37] - is preferable to frequent prompt experimentation in a production setting.

Dataset Design and Ground Truth Construction. When developing evaluation datasets for the search engine, particular care should be taken in how ground truth relevance labels are constructed. During construction of such a dataset, it is recommended that each query be accompanied by an explicit description of the underlying search intent and relevant context. Thomas et al. [37] reported consistently strong performance using this approach.



6.4 Future Work

First, the evaluation should be extended to additional information retrieval datasets. All experiments in this work were conducted on a single benchmark, whose label distribution and annotation quality appear to influence both agreement scores and the relative performance of aggregation strategies. Evaluating the ensemble on datasets with different characteristics would help determine whether the observed differences between aggregation strategies are robust or dataset-specific.

Second, alternative aggregation strategies merit further investigation. This thesis focused on simple, unweighted schemes, namely average voting and majority voting. More informed aggregation methods could leverage estimates of model reliability or capacity. For example, past agreement with human annotations or inter-model agreement statistics could be used to assign weights to individual models. Such approaches may better exploit differences in model quality and reduce the impact of systematically weaker ensemble members.

Third, this work deliberately evaluates an ensemble composed of relatively small models. An important extension would be to study ensembles of medium-sized models that remain economically comparable to the evaluated ensemble, but offer stronger individual performance. Such a configuration could clarify whether the observed limitations are primarily due to aggregation effects or to insufficient model capacity.

Taken together, these directions would help disentangle the effects of dataset characteristics, aggregation design, and model capacity. They would also support more general conclusions about when and how ensemble-based relevance labelling is preferable to single-model approaches.



Chapter 7

Conclusion

This thesis investigated the feasibility of automated relevance labelling using LLMs in the context of donista, with a particular focus on ensemble-based approaches. The work pursued two complementary objectives. First, to assess whether such methods are suitable for practical deployment with respect to label quality, cost, and latency. Second, to examine ensemble-based relevance assessment more broadly and derive insights of relevance to the information retrieval research community.

To this end, a modular relevance labelling pipeline was designed and implemented with a flexible architecture that allows individual components to be exchanged with minimal effort through abstraction into ports and adapters. This design supports systematic experimentation with different models, prompts, and aggregation strategies while remaining suitable for deployment in a production setting. Using this pipeline, an ensemble of smaller LLMs was evaluated relative to other LLM-based judging systems within the LLM Judge Challenge. In addition, its performance was directly compared to that of a strong single-model baseline using agreement with human relevance annotations.

With respect to label quality, the results show that an ensemble of smaller models achieves relevance labelling performance comparable to other contemporary LLM-based judging systems. However, it does not match the inter-rater reliability of a single state-of-the-art model. Agreement with human judgements remains limited for both approaches, with the ensemble exhibiting slightly lower reliability. These findings indicate that aggregating multiple weaker models does not fully compensate for reduced individual model capacity.

In contrast, the ensemble-based approach offers substantial advantages in terms of inference cost. At the evaluated scale, the ensemble reduces cost by a large margin while exhibiting comparable end-to-end latency under sequential execution. These results highlight a clear cost-quality trade-off: a single large model yields higher-quality labels, while an ensemble provides economically attractive labels at lower quality.

Contrary to the initial hypothesis, the ensemble does not reduce the number of hard disagreements. Instead, it produces a higher proportion of severe misclassifications than the reference model. Increasing ensemble size similarly does not lead to consistent improvements in inter-rater reliability. Agreement stabilizes once the ensemble reaches a small number of models, indicating diminishing returns from further expansion. Finally, the results show that aggregation strategy has a considerable impact on label quality. Majority voting outperforms average voting across all evaluated settings, although no universally optimal strategy can be identified.



Beyond the specific ensemble configuration, the results offer a cautious perspective on the LLM-as-a-Judge paradigm as a whole. Agreement between LLMs and human assessors remains relatively low, which limits the strength of agreement-based evaluation alone. Several indicators suggest that annotation noise in the benchmark dataset contributes to this effect, complicating interpretation.

From a practical perspective, the findings support clear recommendations for donista. When label quality is the primary objective and cost is secondary, a single strong model is preferable. When cost efficiency and robustness are important, an ensemble-based approach represents a viable alternative. The implemented pipeline accommodates both approaches and allows switching between them without architectural changes.

Finally, the results of this thesis reflect the capabilities of LLMs available between late 2024 and early 2025 and are shaped by the characteristics of a single benchmark dataset. As models, datasets, and evaluation practices evolve, relative performance and optimal design choices may change. Within these bounds, this work demonstrates that ensemble-based relevance labelling is technically feasible and economically attractive, but not inherently superior in terms of label quality.



Bibliography

- [1] Axel Abels and Tom Lenaerts. “Wisdom from Diversity: Bias Mitigation Through Hybrid Human-LLM Crowds”. In: (May 2025). DOI: 10.48550/ARXIV.2505.12349. arXiv: 2505.12349 [cs.CL] (cited on page 13).
- [2] Payal Bajaj et al. “MS MARCO: A Human Generated Machine Reading Comprehension Dataset”. In: *arXiv preprint arXiv:1611.09268* (2018) (cited on page 33).
- [3] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer International Publishing, 2024. ISBN: 9783031454684. DOI: 10.1007/978-3-031-45468-4 (cited on pages 5–7).
- [4] Chris Buckley et al. “Bias and the limits of pooling”. In: *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*. SIGIR06. ACM, Aug. 2006, pp. 619–620. DOI: 10.1145/1148170.1148284 (cited on page 10).
- [5] Guiming Hardy Chen et al. “Humans or LLMs as the Judge? A Study on Judgement Biases”. In: (Feb. 2024). DOI: 10.48550/ARXIV.2402.10669. arXiv: 2402.10669 [cs.CL] (cited on page 13).
- [6] Zhijun Chen et al. “Harnessing Multiple Large Language Models: A Survey on LLM Ensemble”. In: (Feb. 2025). DOI: 10.48550/ARXIV.2502.18036. arXiv: 2502.18036 [cs.CL] (cited on page 12).
- [7] Alistair Cockburn. *The Hexagonal (Ports & Adapters) Architecture*. Website. accessed 2026-01-04. Sept. 2005. URL: <https://alistair.cockburn.us/hexagonal-architecture> (cited on page 16).
- [8] Alistair Cockburn. *Hexagonal Architecture (Ports and Adapters)*. Talk at Craft-Con 2023. accessed 2025-10-20. May 2023. URL: <https://alistaircockburn.com/Articles/Hexagonal-Architecture-slides-CraftCon-2023-Budapest> (cited on pages 17, 30).
- [9] Nick Craswell et al. “Overview of the TREC 2019 deep learning track”. In: (Mar. 2020). DOI: 10.48550/ARXIV.2003.07820. arXiv: 2003.07820 [cs.IR] (cited on pages 8, 33).
- [10] Laura Dietz et al. “Principles and Guidelines for the Use of LLM Judges”. In: *Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR)*. ICTIR ’25. Padua, Italy: Association for Computing Machinery, July 2025, pp. 218–229. ISBN: 9798400718618. DOI: 10.1145/3731120.3744588 (cited on page 13).
- [11] Margherita Grandini, Enrico Bagli, and Giorgio Visani. “Metrics for Multi-Class Classification: an Overview”. In: (Aug. 2020). DOI: 10.48550/ARXIV.2008.05756. arXiv: 2008.05756 [stat.ML] (cited on pages 40, 41).



- [12] Lu Hong and Scott E. Page. “Groups of Diverse Problem Solvers Can Outperform Groups of High-Ability Problem Solvers”. In: *Proceedings of the National Academy of Sciences* 101.46 (Nov. 2004), pp. 16385–16389. ISSN: 1091-6490. DOI: 10.1073/pnas.0403723101 (cited on pages 12, 13).
- [13] Kasra Hosseini et al. “Retrieve, Annotate, Evaluate, Repeat: Leveraging Multimodal LLMs for Large-Scale Product Retrieval Evaluation”. In: (Sept. 2024). DOI: 10.48550/ARXIV.2409.11860. arXiv: 2409.11860 [cs.IR] (cited on pages 1, 10, 35).
- [14] Kalervo Järvelin and Jaana Kekäläinen. “Cumulated gain-based evaluation of IR techniques”. In: *ACM Transactions on Information Systems* 20.4 (Oct. 2002), pp. 422–446. ISSN: 1558-2868. DOI: 10.1145/582415.582418 (cited on page 9).
- [15] Klaus Krippendorff. *Computing Krippendorff’s Alpha-Reliability*. Website. accessed 2026-01-17. Sept. 2013. URL: <https://www.asc.upenn.edu/sites/default/files/2021-03/Computing%20Krippendorff's%20Alpha-Reliability.pdf> (cited on pages 43–45).
- [16] J. Richard Landis and Gary G. Koch. “The measurement of observer agreement for categorical data”. In: *Biometrics* 33.1 (1977), pp. 159–174. DOI: 10.2307/2529310 (cited on pages 42, 43).
- [17] Haitao Li et al. “LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods”. In: (Dec. 2024). DOI: 10.48550/ARXIV.2412.05579. arXiv: 2412.05579 [cs.CL] (cited on pages 13, 33, 38).
- [18] Tianyang Lin et al. “A Survey of Transformers”. In: (June 2021). DOI: 10.48550/ARXIV.2106.04554. arXiv: 2106.04554 [cs.LG] (cited on page 5).
- [19] Google LLC. *Search Quality Raters Guidelines*. PDF. accessed 2026-01-12. Nov. 2023. URL: <https://developers.google.com/search/blog/2023/11/search-quality-rater-guidelines-update> (cited on page 10).
- [20] LMArena. *Text Arena*. Website. accessed 2026-01-07. Jan. 2026. URL: <https://lmarena.ai/leaderboard/text> (cited on page 34).
- [21] Chiyu Ma et al. “Judging with Many Minds: Do More Perspectives Mean Less Prejudice? On Bias Amplifications and Resistance in Multi-Agent Based LLM-as-Judge”. In: (May 2025). DOI: 10.48550/ARXIV.2505.19477. arXiv: 2505.19477 [cs.AI] (cited on page 39).
- [22] Giacomo Marzi, Marco Balzano, and Davide Marchiori. “K-Alpha Calculator–Krippendorff’s Alpha Calculator: A user-friendly tool for computing Krippendorff’s Alpha inter-rater reliability coefficient”. In: *MethodsX* 12 (June 2024), p. 102545. ISSN: 2215-0161. DOI: 10.1016/j.mex.2023.102545 (cited on page 46).
- [23] Marry L. McHugh. “Interrater reliability: the kappa statistic”. In: *Biochemia Medica* (2012), pp. 276–282. ISSN: 1846-7482. DOI: 10.11613/bm.2012.031 (cited on page 42).
- [24] Ibomoiye Domor Mienye and Theo G. Swart. “Ensemble Large Language Models: A Survey”. In: *Information* 16.8 (Aug. 2025), p. 688. ISSN: 2078-2489. DOI: 10.3390/info16080688. URL: <https://www.mdpi.com/2078-2489/16/8/688> (cited on page 12).
- [25] Tri Nguyen et al. “MS MARCO: A Human Generated Machine Reading Comprehension Dataset”. In: *arXiv preprint arXiv:1611.09268* (2016) (cited on page 33).
- [26] NIST. *TREC 2023*. Website. accessed 2026-01-05. Feb. 2023. URL: <https://trec.nist.gov/data/deep2023.html> (cited on page 34).
- [27] Rising Odegua. “An Empirical Study of Ensemble Techniques (Bagging, Boosting and Stacking)”. In: *Deep Learning IndabaX*. Mar. 2019 (cited on page 12).



- [28] Juan Manuel Garrido de Paz. *bluezone*. Github. accessed 2026-01-04. May 2020. URL: <https://github.com/jmgarridopaz/bluezone/tree/master> (cited on page 17).
- [29] Hossein A. Rahmani. *LLMJudge*. Github. accessed 2026-01-05. 2024. URL: <https://github.com/llm4eval/LLMJudge> (cited on page 34).
- [30] Hossein A. Rahmani et al. “JudgeBlender: Ensembling Judgments for Automatic Relevance Assessment”. In: (Dec. 2024). DOI: 10.48550/ARXIV.2412.13268. arXiv: 2412.13268 [cs.IR] (cited on pages 12, 38, 39, 55, 58).
- [31] Hossein A. Rahmani et al. “Judging the Judges: A Collection of LLM-Generated Relevance Judgements”. In: (Feb. 2025). DOI: 10.48550/ARXIV.2502.13908. arXiv: 2502.13908 [cs.IR] (cited on pages vi, 11, 33, 34, 38).
- [32] Sebastian Raschka, Joshua Patterson, and Corey Nolet. “Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence”. In: *Information* 11.4 (Apr. 2020), p. 193. ISSN: 2078-2489. DOI: 10.3390/info11040193 (cited on page 22).
- [33] Philipp Schoenegger et al. “Wisdom of the silicon crowd: LLM ensemble prediction capabilities rival human crowd accuracy”. In: *Science Advances* 10.45 (Nov. 2024), eadp1528. ISSN: 2375-2548. DOI: 10.1126/sciadv.adp1528 (cited on page 12).
- [34] Ian Soboroff. “Don’t Use LLMs to Make Relevance Judgments”. In: *Information Retrieval Research* 1.1 (Mar. 2025), pp. 29–46. ISSN: 3050-9106. DOI: 10.54195/irrj.19625. URL: <https://irrj.org/article/view/19625> (cited on page 13).
- [35] Marina Sokolova and Guy Lapalme. “A systematic analysis of performance measures for classification tasks”. In: *Information Processing & Management* 45.4 (July 2009), pp. 427–437. ISSN: 0306-4573. DOI: 10.1016/j.ipm.2009.03.002 (cited on page 41).
- [36] James Surowiecki. *The Wisdom of Crowds*. Description based on publisher supplied metadata and other sources. New York: Anchor Books, 2005. 10 pp. ISBN: 978-0-385-72170-7 (cited on page 12).
- [37] Paul Thomas et al. “Large Language Models Can Accurately Predict Searcher Preferences”. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR ’24. Washington DC, USA: Association for Computing Machinery, July 2024, pp. 1930–1940. ISBN: 9798400704314. DOI: 10.1145/3626772.3657707 (cited on pages vi, 1, 3, 10, 11, 13, 38, 40, 53, 55, 61).
- [38] Shivani Upadhyay et al. “UMBRELA: Umbrela is the (Open-Source Reproduction of the) Bing RElevance Assessor”. In: (June 2024). DOI: 10.48550/ARXIV.2406.06519. arXiv: 2406.06519 [cs.IR] (cited on pages 33, 38).
- [39] Ashish Vaswani et al. *Attention Is All You Need*. 2017. DOI: 10.48550/ARXIV.1706.03762 (cited on page 5).
- [40] Pat Verga et al. “Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models”. In: (Apr. 2024). DOI: 10.48550/ARXIV.2404.18796. arXiv: 2404.18796 [cs.CL] (cited on pages 12, 38).
- [41] Ellen M. Voorhees. “TREC: Continuing information retrieval’s tradition of experimentation”. In: *Communications of the ACM* 50.11 (Nov. 2007), pp. 51–54. ISSN: 1557-7317. DOI: 10.1145/1297797.1297822 (cited on pages 8–10).
- [42] Peiyi Wang et al. “Large Language Models are not Fair Evaluators”. In: (May 2023). DOI: 10.48550/ARXIV.2305.17926. arXiv: 2305.17926 [cs.CL] (cited on page 13).
- [43] Wikipedia. *Cohen’s kappa*. accessed 2026-01-07. URL: https://en.wikipedia.org/wiki/Cohen's_kappa (cited on page 42).



- [44] Wayne Xin Zhao et al. *A Survey of Large Language Models*. 2023. DOI: 10.48550/ARXIV.2303.18223 (cited on pages 5, 7, 8).
- [45] Lianmin Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: (June 2023). DOI: 10.48550/ARXIV.2306.05685. arXiv: 2306.05685 [cs.CL] (cited on pages 10, 13).